UNIVERSIDAD DE GUANAJUATO

División de Ingenierías · Campus Irapuato-Salamanca (DICIS)

Manual Técnico del Proyecto Carro FSI (LIDIA)

Arquitectura distribuida, firmware de control y visión, núcleo de navegación en C99,
aplicación Android y evidencia de cierre J16

Base v1.1 · revisión pendiente

12-nov-2026: autonomía con esquive iterativo, NO MEDIDO en el carro. No constituye una publicación v1.2.

Materia:	Fundamentos de los Sistemas de Información (FSI)
Profesor titular:	Dr. Horacio Rostro González
Integrantes del equipo:	Jaime Fernando Guerrero Segoviano Melanie Contreras Solis Vitto Eduardo Rangel Fortuna Azul Abril Cadena Campos
Cierre físico histórico:	10 de noviembre de 2026
Artefactos históricos:	`APP:0.4-6d1b213` · `FW:6d1b213-20261110`
Revisión del repositorio:	`Base v1.1: bd1727b; revisión de autonomía 12-nov-2026` fix/navegacion-esquive-confirmado (pendiente de validación física)

Índice general

1. Introducción y cómo leer el manual **4**

1.1 Propósito del proyecto y arquitectura del sistema . . . 4
1.2 Alcance de la Versión final 1.1 (cierre J16) . . . 4
1.3 Convención de etiquetas de evidencia . . . 4
1.4 Estructura del documento y convención de referencias al código . . . 5

2. Arquitectura general **6**

2.1 Topología de tres nodos y reparto de responsabilidades . . . 6
2.2 Esquema de alimentación eléctrica y separación de fuentes . . . 7
2.3 Defensa en profundidad: cuatro capas de seguridad física y lógica . . . 7

3. Hardware y conexiones **9**

3.1 Plataforma mecánica y etapa de tracción . . . 9
3.2 Electrónica de control, sensores I2C y cámara . . . 9
3.3 Mapa detallado de pines y conexiones . . . 10
3.4 Elementos eléctricos no medidos en el prototipo . . . 11

4. Firmware del carro **12**

4.1 Separación entre hardware (hw_*) y lógica en C99 . . . 12
4.2 Cola de comandos, perfiles PWM y pulso de arranque . . . 12
4.3 Control proporcional-integral de rumbo en rectas (rumbo.c — Cierre J16) . . . 13
4.4 Giroscopio MPU6050, factor de escala en ppm (imu.c) y lazo de giro (giro.c) . . . 14
4.5 Freno láser local (tof.c), watchdog (WD:) y diagnóstico LOOP? . . . 15

5. Cámara **16**

5.1 Hardware de captura y justificación de la resolución HQVGA . . . 16
5.2 Arquitectura de doble núcleo y buzón de último cuadro . . . 16
5.3 Endpoints HTTP y protocolo UDP de celdas (1,244 bytes) . . . 17
5.4 Vigilancia y recuperación automática ante fallos (recuperacion.c) . . . 18

6. Protocolo de comunicación **19**

6.1 Capa de transporte y canales de enlace . . . 19
6.2 Temporización de latidos, sondas y vigilancia . . . 19
6.3 Catálogo completo de comandos y respuestas (incluyendo J16) . . . 19
6.4 Estructura de las tramas SENS y CAPS . . . 21

7. Núcleo de navegación en C **22**

7.1 Por qué se navega por tiempo con giroscopio y cómo funciona la cuadrícula . . . 22
7.2 Modelos de calibración longitudinal (calibracion.c) y angular (calgiro.c) . . . 23
7.3 Máquina de estados, integración en décimas de grado y guiado RUMBO . . . 23
7.4 Esquive iterativo (revisión de autonomía del 12-nov) . . . 24
7.5 Cierre, ayuda y evidencia . . . 25

8. Visión y percepción **26**

8.1 Por qué se fusionan color del piso, red neuronal y sensor láser . . . 26
8.2 Clasificador cromático de piso multiclúster (obstaculos.c) y máscara fija . . . 26
8.3 Detector neuronal EfficientDet-Lite0 en hilo dedicado (ModeloEnHilo.kt) . . . 27
8.4 Corredor de perspectiva (corredor.c) y fusión multimodal (fusion.c) . . . 27

8.5 Sincronización multihilo en JNI (candado_vision — Cierre J16) . . . 28

9. Aplicación Android 29

9.1 Estructura de pantallas y sesiones compartidas . . . 29
9.2 Interpretación de los tres estados de conexión en las capturas . . . 30
9.3 Supervisión de seguridad y controles J16 en la aplicación . . . 31

10. Compilación, instalación y verificación de versión 33

10.1 Herramientas de desarrollo requeridas . . . 33
10.2 Compilación y carga del firmware del carro y de la cámara . . . 33
10.3 Compilación e instalación de la aplicación Android . . . 33
10.4 Artefactos validados de la Versión final 1.1 y verificación criptográfica . . . 34

11. Operación y seguridad física 35

11.1 Lista de verificación previa al encendido . . . 35
11.2 Secuencia de encendido y calibración en reposo . . . 35
11.3 Operación en modo manual y ejecución supervisada de rutas . . . 36
11.4 Jerarquía de paro de emergencia y mantenimiento de baterías . . . 36

12. Pruebas y resultados 37

12.1 Resultados VALIDADO EN EL CARRO . . . 37
12.2 Pruebas deterministas de software en computadora (VERIFICADO) . . . 38
12.3 Resultados SIMULADOS: experimento Monte Carlo J16 (50,000 corridas) . . . 39
12.4 Escenarios y funciones NO MEDIDOS en el carro físico . . . 40
12.5 Autonomía del 12-nov-2026: resultados solo de software . . . 41

13. Límites y trabajo futuro 42

13.1 Límites estructurales de locomoción y odometría . . . 42
13.2 Límites geométricos, visuales y de protocolo . . . 42
13.3 Resumen de elementos NO MEDIDOS y trabajo futuro (PENDIENTE) . . . 43
13.4 Consenso y alcance de la revisión de autonomía . . . 43

14. Proceso de desarrollo 45

14.1 Del control monolítico al desacoplamiento por interfaces en C99 . . . 45
14.2 Creación del núcleo matemático compartido (nucleo_c) y puente JNI . . . 45
14.3 Integración de cámara independiente y percepción multimodal . . . 46
14.4 Sensores embarcados (MPU6050 y VL53L0X) e integración de pose . . . 46
14.5 Cierre de precisión J16 y validación física del esquive en el carro . . . 46

15. Solución de problemas 47

15.1 Método general de aislamiento de fallas . . . 47
15.2 Tabla maestra de diagnóstico por síntoma . . . 47
15.3 Verificación rápida por puerto serie (cuando no hay WiFi disponible) . . . 48

16. Anexos 49

16.1 Glosario técnico . . . 49
16.2 Referencia rápida de comandos y pines . . . 49
16.3 Correspondencia entre capítulos y archivos del repositorio . . . 50
16.4 Lista consolidada de pruebas y mediciones pendientes . . . 51

1. Introducción y cómo leer el manual

Este capítulo presenta el propósito del sistema robótico móvil **LIDIA** (*Carro FSI*), define el alcance funcional de su **Versión final 1.1** y establece las convenciones de lectura que rigen todo el documento. Su objetivo es que cualquier integrante del equipo, docente evaluador o desarrollador futuro pueda distinguir con exactitud qué funciones están comprobadas en el vehículo físico, cuáles provienen de modelos de simulación y cuáles permanecen como mediciones abiertas.

1.1 Propósito del proyecto y arquitectura del sistema

El proyecto **LIDIA** desarrolla un vehículo terrestre a escala con tracción omnidireccional de cuatro ruedas, diseñado para desplazarse bajo control manual o semiautónomo sobre superficies interiores planas, detectar obstáculos frente a su trayectoria y ejecutar maniobras de esquive y reincorporación. El sistema se construyó como proyecto integrador de la materia **Fundamentos de los Sistemas de Información (FSI)** en la División de Ingenierías del Campus Irapuato-Salamanca de la Universidad de Guanajuato.

En lugar de concentrar la adquisición de video, la visión por computadora, la planificación geométrica y el control de potencia en un único microcontrolador de recursos limitados, el diseño reparte el trabajo en tres nodos cooperativos conectados mediante una red inalámbrica local (`CarroFSI`, banda de 2.4 GHz):

1. **Placa de control y sensores en el carro (**`ESP32 DevKit`**,** `192.168.4.1`**):** actúa como punto de acceso WiFi, gobierna los cuatro motores mediante un puente H doble, integra la velocidad angular de un giroscopio `MPU6050`, supervisa la distancia frontal con un sensor láser de tiempo de vuelo `VL53L0X` y ejecuta los lazos locales de seguridad y mantenimiento de rumbo.
2. **Placa de cámara independiente (**`Freenove ESP32-WROVER` **con sensor** `GC0308`**,** `192.168.4.2`**):** captura cuadros en resolución `HQVGA` (`240×176` píxeles) y los entrega directamente al teléfono en formato comprimido (`MJPEG`), crudo (`RGB565`) o resumido en celdas de color por `UDP`.
3. **Teléfono Android con la aplicación** `LIDIA` **(**`APP:0.4-6d1b213`**):** ofrece la interfaz de operación y ejecuta de forma nativa —a través de `JNI` (*Java Native Interface*, puente entre Kotlin y código compilado en C)— el núcleo matemático de navegación, calibración y percepción visual en estándar `C99`.

Esta partición permite que el mismo código en `C99` que toma decisiones dentro del teléfono se compile y verifique en una computadora personal mediante miles de pruebas automatizadas antes de mover el vehículo físico.

1.2 Alcance de la Versión final 1.1 (cierre J16)

La **Versión final 1.1** consolida el trabajo acumulado desde la primera entrega etiquetada (`v1.0`, commit `eb88151`) hasta el cierre de precisión e integración **J16** fusionado en `main`. Los artefactos validados que corresponden a esta edición son:

- **Aplicación Android:** `carro_fsi_0.4-6d1b213.apk` (versión `APP:0.4-6d1b213`, `14,881,402` B, SHA-256 `C5B36FEC74BFEDE9798B4BE7C992D7FEA8369F456DA4D4A10D61A4BBB25ED02F`).
- **Firmware del carro:** `FW:6d1b213-20261110` (`1,034,112` B, SHA-256 `71D671A4E64E0F234015E8685D4E741B87471534E3E50869EE023C5A657B79FC`; solo código fuente y hash, sin publicar el `.bin` por seguridad WiFi).
- **Suites de verificación en PC:** 22 programas y `1,655` casos en `nucleo_c`, 5 programas y `540` casos en `pruebas_firmware`, 54 en el servidor falso y 18 en el verificador de versiones (`0` fallos).

1.3 Convención de etiquetas de evidencia

Para evitar ambigüedades entre lo que el código calcula en un modelo matemático y lo que el vehículo ejecuta sobre el piso real, este manual utiliza una clasificación estricta de cuatro niveles de evidencia. Esta convención se define aquí **una sola vez** y se aplica de manera uniforme en todos los capítulos, tablas y pies de figura:

Etiqueta	Significado técnico en este manual	Criterio de aplicación y alcance en la versión 1.1
VALIDADO EN EL CARRO *(o VERIFICADO)*	Comportamiento, parámetro o resultado comprobado experimentalmente en el vehículo físico o mediante ejecución determinista de las suites de software del repositorio.	En pruebas físicas de ruta del cierre J16 en el carro, **solo el esquive con reincorporación B6** (6 corridas el 10-nov-2026: 0 toques, 5 completas autónomas con error lateral aproximado de ~0 a ~10 cm y 1 en AYUDA sin causa identificada). En software, las 2,267 aserciones automatizadas en PC.
SIMULADO	Resultado obtenido en el simulador físico-matemático del núcleo en C (simulador_navegador.c) bajo perturbaciones estocásticas con semilla fija.	Las 50,000 corridas Monte Carlo (1,000 por escenario en 5 configuraciones) de recta de 150 cm, ruta en L, cuadrado de 60 cm y esquives sintéticos. No sustituye pruebas en piso.
NO MEDIDO	Función implementada en el código y verificada en pruebas lógicas, pero sin corridas físicas registradas en el carro en este cierre.	Rectas físicas con y sin RUMBO, ruta en L, cuadrado de 60 cm, corte de WiFi en marcha, freno autónomo aislado por láser, caja ancha de 40×20 cm, calibración en piso de IMU_ESCALA, circuito GPIO 2 y consumo eléctrico en amperios.
PENDIENTE	Mejora de hardware o software identificada como trabajo futuro que aún no forma parte de la implementación validada.	Incorporación de odometría por codificadores (*encoders*), compensación de deslizamiento lateral de ruedas mecanum y medición instrumentada de corriente.

Tabla 1. *Resumen estructurado (Etiqueta · Significado técnico en este manual).*

Regla de lectura sobre mediciones físicas

Cada vez que se citan los resultados de la validación física del escenario **B6** en el carro, debe tenerse presente que las desviaciones laterales finales respecto a la línea recta original (~10 cm, ~10 cm, ~5 cm, ~0 cm y ~0 cm en las cinco corridas autónomas completadas) son **medidas aproximadas** observadas en pista al concluir la maniobra. Asimismo, ninguna afirmación de este manual debe interpretarse como que las rectas con control de rumbo, la ruta en L o el cuadrado están validados físicamente en el carro: esos tres escenarios son SIMULADOS en computadora y permanecen NO MEDIDOS en el vehículo real.

1.4 Estructura del documento y convención de referencias al código

El manual está organizado en dieciséis capítulos agrupados en cuatro bloques lógicos:

- **Arquitectura y plataforma física (Capítulos 2 a 5):** describe la topología de tres nodos, el esquema eléctrico con dos baterías USB independientes más el portapilas de tracción, la electrónica del chasis, el firmware de control en la ESP32 DevKit y el servidor de visión en la cámara Freenove.
- **Comunicación, algoritmos e interfaz (Capítulos 6 a 9):** detalla el protocolo de texto sobre WebSocket, el núcleo matemático de navegación en C99, la tubería multimodal de percepción visual y el diseño de la aplicación Android.
- **Ingeniería de entrega, operación y evidencia (Capítulos 10 a 13):** documenta el flujo de compilación y verificación criptográfica de versiones, los procedimientos seguros de operación, los resultados experimentales y simulados, y los límites físicos conocidos.
- **Evolución, diagnóstico y consulta rápida (Capítulos 14 a 16):** sintetiza la evolución técnica del proyecto, ofrece una guía sistemática de solución de problemas e incluye los anexos de glosario, comandos, pines, correspondencia completa de archivos del repositorio y lista de tareas abiertas.

Para mantener una lectura fluida, el cuerpo de los capítulos no intercala rutas de carpetas ni números de línea. Al final de cada capítulo del 1 al 15 se incluye una sola línea titulada **Dónde está en el código** con los nombres de los archivos fuente principales, mientras que el **Anexo 16.3** presenta la tabla completa con las rutas exactas dentro del repositorio.

Dónde está en el código: README.md, entrega.md, estado_20261110.md, limites_conocidos.md.

2. Arquitectura general

Este capítulo explica cómo se divide el sistema en tres nodos físicos independientes, por qué se eligió esta distribución frente a un diseño centralizado en el microcontrolador y cómo se protegen eléctricamente y lógicamente sus enlaces. Su comprensión permite ubicar el papel de cada placa, cada batería y cada protocolo dentro del conjunto.

2.1 Topología de tres nodos y reparto de responsabilidades

El vehículo opera mediante la cooperación de tres nodos que se comunican sobre una red inalámbrica local (`SSID: CarroFSI`) creada por la placa principal del chasis. Ningún nodo depende de internet ni de servidores externos.

Nodo del sistema	Dirección IP y rol de red	Responsabilidad técnica principal	Fuente de energía dedicada
ESP32 DevKit (control y sensores)	`192.168.4.1` — Punto de acceso (AP), servidor `HTTP :80` y `WebSocket /ws`	Cola determinista de comandos, PWM de motores (`1 kHz`), lectura `I2C` a `400 kHz` (`MPU6050` y `VL53L0X`), lazo `PI` de rumbo en rectas (`RUMBO`), giro cerrado (`GIRO`), freno láser local (`120 mm`) y *watchdogs* (`1000 ms` / `5000 ms`).	**Powerbank USB #1** (`5 V` por puerto micro-USB; alimenta también el bus `I2C` a `3.3 V`).
Freenove ESP32-WROVER (cámara GC0308)	`192.168.4.2` — Cliente (STA) unido a `CarroFSI`; servidores `HTTP :80/:81` y `UDP :4210`	Captura continua en resolución `HQVGA` (`240×176` píxeles, `RGB565` a `20 MHz`), compresión `JPEG` por software (calidad `50`), buzón de último cuadro y envío de paquetes de celdas (`1,244 B`).	**Powerbank USB #2** (`5 V` exclusiva para la cámara; aislada galvánicamente del chasis).
Teléfono Android (AP P:0.4-6d1b213)	`192.168.4.x` — Cliente (STA) con sesiones persistentes `SesionCarro` y `SesionVideo`	Interfaz de usuario, ejecución nativa `JNI` del núcleo en `C99` (planificación en cuadrícula de `20 cm`, integración de pose en décimas de grado, clasificador cromático `YCbCr`, inferencia `EfficientDet-Lite0`, fusión bajo mutex) y supervisión activa de seguridad.	Batería interna del teléfono móvil.

Tabla 2. *Resumen estructurado (Nodo del sistema · Dirección IP y rol de red).*

*(En el cuadro de tracción, los cuatro motores DC de caja reductora `TT` y la etapa de potencia del puente H `TC1508A` reciben energía de un **portapilas independiente de 4 pilas AA** con interruptor físico, unido por tierra común `GND` a la `ESP32 DevKit`).*

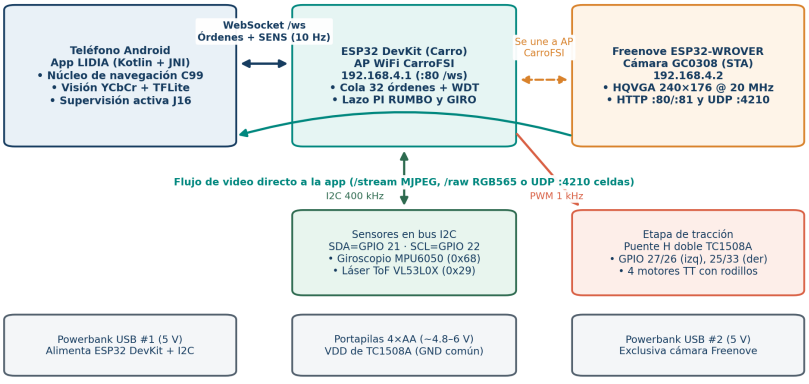


Figura 1. *Arquitectura distribuida de tres nodos del sistema LIDIA, enlaces de red e interfaces de hardware.*

Por qué el teléfono ejecuta el cálculo pesado y la cámara va en otra placa

En un microcontrolador `ESP32` de doble núcleo a `240 MHz`, capturar video de un sensor sin compresión nativa (`GC0308`), comprimir cada cuadro a `JPEG` por software, ejecutar una red neuronal de detección de objetos y atender simultáneamente una pila `WiFi/TCP` consume decenas de milisegundos por ciclo e introduce latencias variables (*jitter*). Si ese mismo procesador tuviera que integrar el giroscopio a `200 Hz` y cerrar el lazo de rumbo a `50 Hz`, cualquier ráfaga de red o de compresión de imagen degradaría la precisión angular y retrasaría el frenado ante un obstáculo.

Separar las funciones resuelve este conflicto de raíz:

1. La **ESP32 DevKit** queda dedicada a tareas de tiempo real estricto: su bucle principal dura en promedio ~0.69 ms (690 us) y solo se alarga durante las lecturas del sensor láser VL53L0X, sin sufrir bloqueos por transmisión de imágenes.
2. La **Freenove ESP32-WROVER** dedica sus dos núcleos y su memoria PSRAM exclusivamente a leer el sensor GC0308 y servir el cuadro más reciente al teléfono.
3. El **teléfono Android** aporta procesadores multinúcleo de varios gigahercios y aceleración vectorial para ejecutar el clasificador de piso, la red neuronal TensorFlow Lite y la máquina de estados del navegador en C99, enviando al carro únicamente consignas compactas (FORWARD, RUMBO:<décimas>, GIRO:<grados>, STOP).

2.2 Esquema de alimentación eléctrica y separación de fuentes

Uno de los problemas clásicos en robótica móvil a escala es la caída de tensión (*brownout*) cuando los motores arrancan o invierten su sentido de giro: el pico de corriente hace caer el voltaje de la batería y reinicia el microcontrolador o desestabiliza la cámara. Para impedirlo, el vehículo divide su alimentación en **tres suministros físicamente separados**:

1. **Powerbank USB #1 (electrónica de control):** alimenta por USB a la placa ESP32 DevKit. Desde el regulador interno de 3.3 V de esta placa se energizan únicamente el giroscopio MPU6050 (0x68) y el sensor láser VL53L0X (0x29).
2. **Portapilas de 4 pilas AA (potencia de tracción):** entrega entre 4.8 V (con cuatro celdas recargables NiMH de 1.2 V) y 6.0 V (con celdas alcalinas de 1.5 V) directamente al pin VDD del puente H doble TC1508A a través de un interruptor mecánico en el chasis. El positivo de las pilas **nunca** se conecta a ningún pin de la ESP32 DevKit.
3. **Powerbank USB #2 (exclusiva de la cámara):** alimenta por su propio cable USB a la placa Freenove ESP32-WROVER. La cámara no tiene ningún cable de señal ni de tierra unido al resto del carro; su acoplamiento es cien por ciento inalámbrico por WiFi.

Para que las cuatro señales PWM (GPIO 27, 26, 25 y 33) que salen de la ESP32 DevKit hacia las entradas IN1..IN4 del driver TC1508A tengan una referencia eléctrica válida, los terminales negativos (GND) de la ESP32 DevKit, del módulo TC1508A y del portapilas de 4×AA están unidos en un bus de **tierra común**.

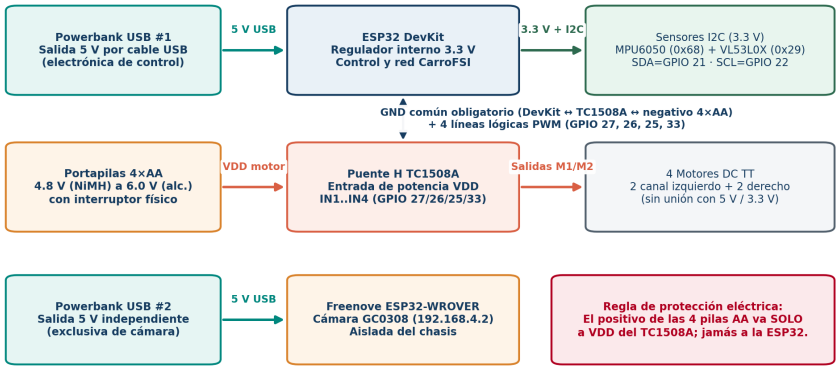


Figura 2. *Esquema de los tres circuitos de alimentación (dos powerbanks USB independientes más el portapilas de 4 pilas AA) y el enlace de tierra común.*

2.3 Defensa en profundidad: cuatro capas de seguridad física y lógica

Dado que los algoritmos de percepción y navegación corren en el teléfono mientras los motores están en el carro, cualquier pérdida de enlace WiFi, bloqueo de aplicación o lectura espuria podría dejar al vehículo avanzando sin control si no existieran barreras locales. El sistema implementa cuatro anillos concéntricos de seguridad:

1. **Capa 1 — Supervisión humana e interruptor físico:** el operador mantiene contacto visual con el vehículo, dispone de un botón permanente PARO en la interfaz y puede cortar la energía de tracción desde el interruptor del portapilas 4×AA.

2. **Capa 2 — Supervisión activa en Android (reforzada en J16):** durante la conducción manual, `ControlLogica` envía un latido de movimiento cada `250 ms` mientras se pulsa un botón, pero **corta inmediatamente el reenvío de** `FORWARD` si recibe `TOF_PARO` del carro. Durante una ruta semiautónoma, `ModoSemiautonomo` aborta la misión y envía `STOP` en tres situaciones adicionales incorporadas en el cierre J16:
- Si recibe el aviso `WD:<n>` indicando que el *watchdog* del carro se disparó por falta de comandos.
- Si transcurren más de `1,500 ms` (`SILENCIO_SENS_MS`) sin recibir tramas `SENS` mientras el navegador está activo.
- Si llega `TOF_PARO` y la máquina de estados del navegador no transiciona a parada o esquive en ese mismo ciclo.
3. **Capa 3 — Vigilancia temporal en el firmware (`ESP32 DevKit`):** todos los mensajes entrantes pasan por una cola determinista de 32 ranuras donde las órdenes `STOP` y `EBRAKE` tienen prioridad absoluta y jamás se descartan (si la cola se llena, desaloja un `PING` antiguo). Si pasan `1,000 ms` (`TIMEOUT_COMANDO_MS`) con los motores encendidos sin recibir una orden válida de movimiento, `carro_vigilar_comunicacion` apaga los motores, restablece el perfil `AUTO` y transmite `WD:<contador>`. Además, un *watchdog* de hardware del ESP32 (`WDT_HARDWARE_SEG = 5 s`) reinicia el microcontrolador a estado seguro (`STOP`) si el bucle principal llegara a bloquearse.
4. **Capa 4 — Paro reflejo por sensor láser frontal (`VL53L0X`):** cuando el carro avanza hacia el frente en perfil `AUTO` y el sensor láser mide una distancia válida inferior a `120 mm` (`TOF_PARO_MM`), el propio firmware detiene los motores en menos de un ciclo de sensores (`~20 ms`), sin esperar instrucción del teléfono, y notifica el evento mediante `TOF_PARO:<mm>`.

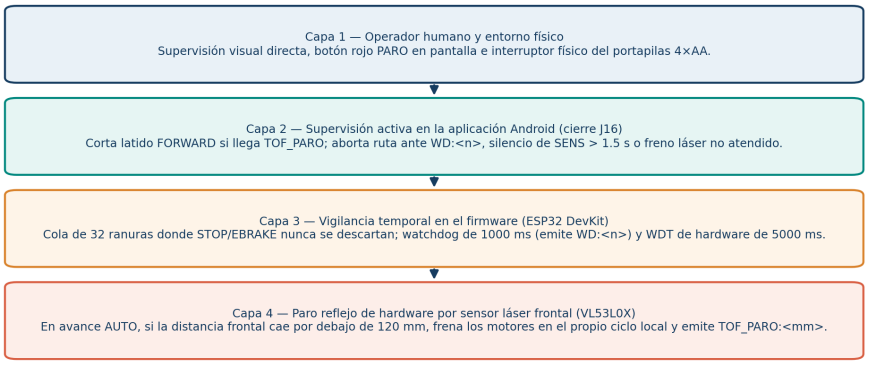


Figura 3. *Defensa en profundidad de cuatro capas: operador, supervisión activa en Android, vigilancia temporal en firmware y paro reflejo por láser.*

Dónde está en el código: `config.h`, `carro.c`, `sensores.c`, `ControlLogica.kt`, `ModoSemiautonomo.kt`.

3. Hardware y conexiones

Este capítulo describe los componentes mecánicos, electrónicos y sensores que integran el vehículo físico, así como el mapa exacto de conexiones entre la placa `ESP32 DevKit`, el puente H `TC1508A` y el bus `I2C`. También deja asentado qué circuitos auxiliares están solamente declarados en software pero permanecen sin medición física en el prototipo actual.

3.1 Plataforma mecánica y etapa de tracción

El vehículo utiliza un chasis rígido de cuatro ruedas omnidireccionales (*mecanum*) acopladas a cuatro motoreductores de corriente directa tipo **TT** (relación de reducción típica 1:48, tensión nominal de `3 V` a `6 V`). A diferencia de un vehículo con dirección tipo Ackermann, la orientación del carro se modifica por **tracción diferencial** entre el costado izquierdo y el costado derecho:

- **Avance recto (`FORWARD`):** los dos motores izquierdos y los dos motores derechos giran hacia adelante.
- **Retroceso (`BACKWARD`):** ambos costados giran hacia atrás.
- **Giro sobre su propio eje (`LEFT` / `RIGHT`):** un costado gira hacia adelante y el costado opuesto gira hacia atrás (pivote diferencial).
- **Freno activo (`EBRAKE`):** ambos terminales de cada canal se ponen en alto (`HIGH` / `HIGH`) para cortocircuitar la fuerza contraelectromotriz del inducido y detener el giro con rapidez.
- **Paro libre (`STOP`):** ambos terminales se ponen en bajo (`LOW` / `LOW`), cortando toda corriente hacia los motores.

Como el puente H empleado (`TC1508A`) dispone de **dos canales de potencia** (y no de cuatro canales independientes), los dos motores del lado izquierdo están conectados en paralelo al **Canal 1 (`OUT1` / `OUT2`)** y los dos motores del lado derecho están conectados en paralelo al **Canal 2 (`OUT3` / `OUT4`)**. Por esta razón eléctrica, aunque las llantas físicas tienen rodillos oblicuos tipo *mecanum*, el prototipo opera cinemáticamente como un vehículo diferencial de cuatro ruedas motrices y no puede realizar desplazamiento lateral puro (*strafe*) sin girar.

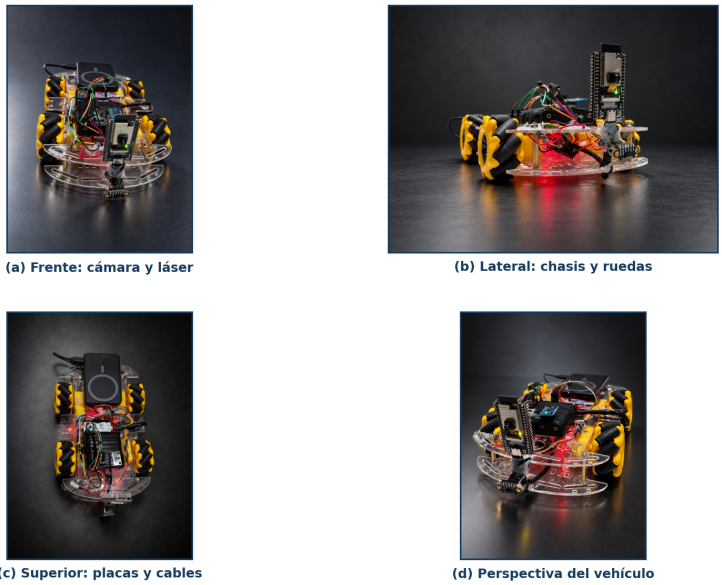


Figura 4. *Vistas físicas del vehículo LIDIA: (a) frente con cámara y sensor láser, (b) distribución superior de placas y baterías, (c) chasis y ruedas, (d) cableado en protoboard.*

3.2 Electrónica de control, sensores I2C y cámara

El inventario electrónico embarcado sobre el chasis consta de cinco módulos principales:

Componente	Modelo / Especificación	Interfaz y dirección / pines	Función en el vehículo	Estado de evidencia
Microcontrolador principal	ESP32 DevKit V1 (ESP-WROOM-32, 240 MHz, WiFi 2.4 GHz)	PWM: GPIO 27, 26, 25, 33 I2C: GPIO 21 (SDA), 22 (SCL)	Punto de acceso CarroFSI (192.168.4.1), control de tracción, lectura de sensores y seguridad local.	VALIDADO EN EL CARRO
Driver de motores	Puente H doble TC1508A (reemplazo funcional de L298N en baja tensión)	IN1=27, IN2=26 (izq.) IN3=25, IN4=33 (der.) VDD a 4×AA; GND común	Conmuta la corriente del portapilas 4×AA hacia los 4 motores TT con modulación PWM a 1,000 Hz (8 bits, 0..255).	VALIDADO EN EL CARRO
Giroscopio MEMS	MPU6050 (6 ejes; se emplea el eje Z de velocidad angular ω_z)	Bus I2C a 400 kHz Dirección 0x68 Alimentación 3.3 V	Mide velocidad de guiñada en el rango de ±500 °/s (65.5 LSB/(°/s)), muestreado a 200 Hz para giros y rumbo.	VALIDADO EN EL CARRO
Sensor de distancia	Láser de tiempo de vuelo (*ToF*) VL53L0X (940 nm)	Bus I2C a 400 kHz Dirección 0x29 Alimentación 3.3 V	Mide distancia frontal en milímetros (ventana válida 20..2000 mm) cada 20 ms; dispara TOF_PARO bajo 120 mm.	VALIDADO EN EL CARRO
Módulo de cámara	Freenove ESP32-WROVER con cámara GC0308 y PSRAM	Enlace WiFi (192.168.4.2) D0..D7, XCLK=21, SIOD=26, SIOC=27 internos	Captura imágenes HQVGA (240×176 px) y las sirve por HTTP (:80/:81) o UDP (:4210) al teléfono.	VALIDADO EN EL CARRO

Tabla 3. *Resumen estructurado (Componente · Modelo / Especificación).*

3.3 Mapa detallado de pines y conexiones

Todas las asignaciones de pines del microcontrolador de control están centralizadas en `config.h`. La siguiente tabla resume cada conexión física del chasis:

Pin en ESP32 DevKit	Constante en firmware	Destino físico	Nivel / Protocolo	Observaciones de montaje
GPIO 27	PIN_MOTOR_IZQ_A	TC1508A IN1	PWM 1000 Hz (canal 0)	Avance del par de motores izquierdos.
GPIO 26	PIN_MOTOR_IZQ_B	TC1508A IN2	PWM 1000 Hz (canal 1)	Retroceso del par de motores izquierdos.
GPIO 25	PIN_MOTOR_DER_A	TC1508A IN3	PWM 1000 Hz (canal 2)	Avance del par de motores derechos.
GPIO 33	PIN_MOTOR_DER_B	TC1508A IN4	PWM 1000 Hz (canal 3)	Retroceso del par de motores derechos.
GPIO 21	PIN_I2C_SDA	SDA de MPU6050 y VL53L0X	I2C 400 kHz (3.3 V)	Línea de datos compartida en protoboard.
GPIO 22	PIN_I2C_SCL	SCL de MPU6050 y VL53L0X	I2C 400 kHz (3.3 V)	Línea de reloj compartida en protoboard.
GPIO 2	PIN_CARGA_POWERBANK	LED interno / carga opcional	Salida digital (3.3 V)	Pulso de 200 ms cada 8,000 ms; circuito externo y efecto real NO MEDIDOS.
3V3	—	VCC de MPU6050 y VL53L0X	Alimentación regulada 3.3 V	Exclusivo para sensores de bajo consumo.
GND	—	GND de TC1508A, sensores y - de 4×AA	Referencia común (0 V)	Imprescindible para que el TC1508A interprete los pulsos PWM.

Tabla 4. *Resumen estructurado (Pin en ESP32 DevKit · Constante en firmware).*

Orientación física de los sensores en el chasis

1. **Giroscopio (`MPU6050`):** va montado horizontalmente cerca del centro geométrico del chasis. Dado que según la cara en que se fije la placa sobre la protoboard el eje `Z` puede apuntar hacia arriba o hacia abajo, el firmware incluye la constante `IMU_SIGNO_Z = 1` (o `-1`) para garantizar la convención de signos de todo el sistema: **giro hacia la izquierda positivo (`+θ`)** y **giro hacia la derecha negativo (`-θ`)**.
2. **Láser frontal (`VL53L0X`):** va colocado en el borde delantero del chasis apuntando horizontalmente en la dirección de avance (`0°`), por debajo del lente de la cámara `Freenove`, de modo que detecte cajas de `20×20 cm` situadas frente al vehículo.

3.4 Elementos eléctricos no medidos en el prototipo

Siguiendo la regla de trazabilidad del manual, dos aspectos eléctricos deben declararse explícitamente como `NO MEDIDOS`:

- **Circuito de mantenimiento de *powerbank* en `GPIO 2`:** algunas baterías portátiles comerciales apagan su salida USB si el consumo cae por debajo de un umbral durante varios segundos. El firmware conmuta `GPIO 2` en alto durante `200 ms` cada `8,000 ms` (`CARGA_PERIODO_MS = 8000`, `CARGA_PULSO_MS = 200`), pensando en conmutar un resistor de carga hacia tierra. Sin embargo, el consumo de la `ESP32 DevKit` con el radio `WiFi` activo en modo punto de acceso (`AP`) basta por sí mismo para mantener encendida la *powerbank* empleada en las pruebas, por lo que la eficacia del pulso en `GPIO 2` permanece `NO MEDIDA`.
- **Consumo de corriente y autonomía en miliamperios-hora:** no se han efectuado mediciones instrumentadas con multímetro o pinza amperimétrica de la corriente de arranque ni de régimen de los cuatro motores `TT`, ni del consumo exacto de las placas `ESP32`. En su lugar, la operación se rige por el criterio experimental de reemplazo o recarga de las `4 pilas AA` cada **20 corridas o 90 minutos de uso** (descrito en el Capítulo 11).

Dónde está en el código: `config.h`, `firmware_carro.ino`, `camara_prueba.ino`.

4. Firmware del carro

Este capítulo detalla la arquitectura interna del programa que corre en la placa ESP32 DevKit (FW:6d1b213-20261110), su separación respecto al hardware físico mediante interfaces en C99, y el funcionamiento matemático de sus lazos de control y seguridad. Incluye de manera íntegra las innovaciones del cierre **J16**: el controlador proporcional-integral de rumbo en rectas (rumbo.c), el factor de escala del giroscopio en partes por millón (imu.c), la normalización de freno en giros largos (giro.c) y el aviso de *watchdog* (WD:<n>).

4.1 Separación entre hardware (hw_*) y lógica en C99

El código del firmware está dividido deliberadamente en dos estratos:

1. **Capa de plataforma (**firmware_carro.ino**):** es el único archivo que conoce las bibliotecas de Arduino y del ESP32 (WiFi.h, AsyncTCP.h, ESPAsyncWebServer.h, Wire.h, Preferences.h, esp_task_wdt.h). Configura el punto de acceso CarroFSI, atiende el servidor WebSocket en /ws, inicializa los periféricos PWM e I2C e implementa las funciones puente con prefijo hw_* (hw_aplicar_motores, hw_millis, hw_micros, hw_imu_leer_gz, hw_tof_leer_mm, hw_nvs_guardar_escala_imu, hw_enviar_texto, etc.).
2. **Módulos lógicos en** C99 **puro (**carro.c**,** sensores.c**,** imu.c**,** tof.c**,** giro.c**,** rumbo.c**):** no incluyen <Arduino.h> ni tocan registros físicos; toda su interacción con el exterior ocurre invocando las funciones hw_*. Gracias a este diseño, los seis módulos se compilan en computadora personal con gcc y se someten a **540 casos de prueba unitarios e integrados** (pruebas_firmware) simulando escenarios extremos (saturación de cola, fallos de bus I2C, ruido de giroscopio, bloqueo de láser o pérdida de WiFi) sin necesidad de energizar el vehículo.

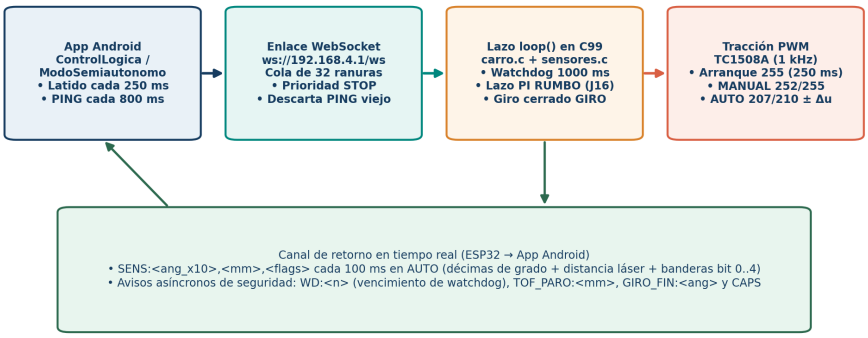


Figura 5. *Flujo de órdenes desde la aplicación hasta los motores a través de la cola determinista y retorno de telemetría SENS y avisos de seguridad.*

4.2 Cola de comandos, perfiles PWM y pulso de arranque

En el ESP32, los eventos de red de ESPAsyncWebServer se ejecutan en una tarea de fondo (async_tcp) distinta de la tarea donde corre loop(). Si el callback de red modificara directamente los pines PWM o el estado de los controladores mientras loop() actualiza los sensores, se producirían condiciones de carrera.

Para evitarlo, el callback de WebSocket solo invoca carro_encolar_comando, que deposita el mensaje en una cola circular protegida por sección crítica de **32 ranuras de 32 bytes** (COLA_COMANDOS_CAP = 32). En cada iteración de loop(), carro_procesar_cola extrae y ejecuta los mensajes en orden FIFO con una **regla de prioridad de seguridad**:

- Si la cola está llena y llega un comando normal, se descarta el mensaje entrante e incrementa el contador de descartes.
- Si la cola está llena y llega STOP **o** EBRAKE, el firmware busca en la cola un mensaje PING antiguo y lo reemplaza; si no hay ninguno, sobrescribe la última ranura para garantizar que **ninguna orden de paro se pierda jamás**.

Perfiles de potencia y compensación de asimetría mecánica

El firmware ofrece dos perfiles de ciclo de trabajo PWM (rango de 8 bits: 0 a 255):

Parámetro	Constante en config.h	Valor	Propósito y estado de validación
Perfil MANUAL izquierdo	PWM_MANUAL_IZQ	252 (98.8 %)	Calibrado físicamente en pista: redujo el desvío lateral en recta de 150 cm de 14.5 cm (con 255/255) a 3.0 cm (VALIDADO EN EL CARRO).
Perfil MANUAL derecho	PWM_MANUAL_DER	255 (100.0 %)	Referencia máxima para respuesta ágil con botones manuales.
Perfil AUTO izquierdo	PWM_AUTO_IZQ	207 (81.2 %)	Usado en rutas semiautónomas (VALIDADO EN EL CARRO dentro del esquive B6; su balance aislado en recta libre permanece NO MEDIDO).
Perfil AUTO derecho	PWM_AUTO_DER	210 (82.4 %)	Velocidad moderada (~82 %) que da margen superior e inferior al control RUMBO y tiempo de reacción a la cámara y al láser.
Pulso de arranque	PWM_ARRANQUE / TIEMPO_ARRANQUE_MS	255 durante 250 ms	Rompe la fricción estática de las cuatro cajas reductoras TT al pasar de reposo a movimiento (evita que los motores zumben sin girar).

Tabla 5. *Resumen estructurado (Parámetro · Constante en config.h).*

4.3 Control proporcional-integral de rumbo en rectas (rumbo.c — Cierre J16)

Aunque el ajuste estático de PWM (252/255 o 207/210) corrige la asimetría media de los motores en una condición dada, el rozamiento desigual de los rodillos *mecanum*, las juntas del piso y la descarga gradual de las pilas AA provocan que el carro tienda a curvarse en tramos largos. Para corregir esta deriva en tiempo real sin necesidad de detenerse, el cierre **J16** incorporó el módulo rumbo.c, que ejecuta un controlador **proporcional-integral (PI)** a 50 Hz (SENS_CONTROL_MS = 20 ms) durante cualquier orden FORWARD en perfil AUTO cuando el control de rumbo está habilitado (RUMBO:ON, activo por defecto al iniciar).

Sea θ_ref la referencia de orientación en décimas de grado (fijada automáticamente al ángulo actual al iniciar FORWARD, o actualizada al vuelo por el teléfono mediante RUMBO:<décimas>) y sea θ el ángulo integrado del giroscopio en décimas de grado. El error angular envuelto al intervalo [-1800, +1800] décimas de grado es:

$$e_x10 = envolver_1800(\theta_ref - \theta)$$

Como en la convención del vehículo un ángulo positivo (+θ) corresponde a girar hacia la **izquierda**, un error e_x10 > 0 significa que el carro se ha desviado hacia la **derecha** (θ < θ_ref) y necesita girar hacia la izquierda: es decir, **frenar ligeramente el lado izquierdo y acelerar el lado derecho**. El controlador calcula el ajuste diferencial en milésimas del PWM base (ajuste_mil) mediante tres términos:

1. **Banda muerta (RUMBO_BANDA_MUERTA_X10 = 2, es decir, 0.2°):** si |e_x10| ≤ 2, el error instantáneo se toma como cero para no excitar oscilaciones por ruido del sensor.
2. **Término proporcional (P):** con ganancia RUMBO_KP_MIL_GRADO = 10 (1.0 % de ajuste PWM por cada grado de error):

$$P_mil = (10 \cdot e_ef) / (10) = e_ef$$

3. **Término integral (I) con anti-*windup* y aprendizaje entre tramos:** con constante de tiempo RUMBO_TI_MS = 1500 ms, acumula integral_x10_ms (acotada para no superar por sí sola el tope de salida):

$$I_mil = (10 \cdot integral_x10_ms) / (10 \cdot 1500)$$

4. **Sesgo aprendido persistente entre tramos (`aprendido_mil`):** cuando termina un tramo de avance (`rumbo_detener`), el valor alcanzado por el integrador `I_mil` no se pierde: se transfiere al acumulador `aprendido_mil` (acotado a ±120 milésimas, ±12 %). Así, en el siguiente tramo recto de la misma ruta, el carro arranca aplicando desde el primer milisegundo la compensación de asimetría que ya descubrió para ese piso y esas baterías.

La salida total `u_mil = clamp_[-150, +150](aprendido_mil + P_mil + I_mil)` se limita a ±150 milésimas (±15 %, `RUMBO_MAX_MIL = 150`) y modifica simétricamente los ciclos de trabajo base (`B_L = 207`, `B_R = 210`):

```
PWM_izq = clamp_[140, 255](B_L - round((B_L · u_mil) / (1000))), PWM_der = clamp_[140, 255](B_R + round((B_R · u_mil) / (1000)))
```

Ejemplo numérico del controlador PI de rumbo

Supóngase que el carro avanza en perfil `AUTO` (`B_L = 207`, `B_R = 210`) con referencia `θ_ref = 0.0°` y el giroscopio detecta que el chasis se desvió `3.0°` hacia la derecha (`θ = -3.0° ⇒ e_x10 = +30`), tras mantener ese desvío durante `0.6 s` (`600 ms`) con `aprendido_mil = 0`:

- El término proporcional aporta `P_mil = +30` milésimas (+3.0 %).
- El integrador acumula `30 × 600 = 18,000 décimas·ms`, por lo que `I_mil = 18,000 / 1,500 = +12` milésimas (+1.2 %).
- El ajuste total es `u_mil = 30 + 12 = +42` milésimas (+4.2 %).
- El motor izquierdo reduce su PWM a `207 - round(207 × 42 / 1000) = 207 - 9 = 198`, mientras el motor derecho aumenta su PWM a `210 + round(210 × 42 / 1000) = 210 + 9 = 219`, generando un par corrector suave que devuelve el carro a `0.0°`.

Además, si el operador o el navegador cambia la referencia con `RUMBO:<décimas>` en más de 1.5° (`RUMBO_UMBRAL_RECTO_X10 = 15`) durante los primeros 250 ms de avance, `sensores_aplicar_motores_avance` **cancela de inmediato el pulso de arranque a** 255 para poder aplicar la diferencia de PWM sin esperar a que termine el cuarto de segundo inicial.

4.4 Giroscopio `MPU6050`, factor de escala en ppm (`imu.c`) y lazo de giro (`giro.c`)

El módulo `imu.c` muestrea el registro de velocidad angular en Z del `MPU6050` cada `5 ms` (`200 Hz`, filtro pasa-bajas digital interno `DLPF` a `42 Hz`, escala de `±500 °/s` con sensibilidad nominal de `65.5 LSB/(°/s)`). Al encender o al recibir el comando `CAL_IMU`, el firmware exige que el carro permanezca inmóvil durante `100` muestras (`~500 ms`); si la dispersión pico a pico no supera `80` LSB (`~1.2 °/s`), promedia las lecturas para fijar el sesgo en reposo (`bias`).

Factor de escala en partes por millón (`IMU_ESCALA` — Cierre J16)

Los giroscopios MEMS de bajo costo presentan una tolerancia de sensibilidad típica de ±1 % a ±3 % respecto al valor nominal de hoja de datos (`65.5 LSB/(°/s)`). En un giro único de 90°, un error de escala del 2 % equivale a apenas 1.8°, pero tras acumular cuatro giros en un cuadrado o múltiples maniobras de esquive, el error sistemático se suma hasta alcanzar de 7° a 10°.

Para corregirlo sin usar aritmética de punto flotante lenta ni perder resolución, el cierre **J16** añadió en `imu.c` el parámetro `escala_ppm`, expresado en **partes por millón** dentro del intervalo seguro `[-50000, +50000]` (±5 %) y persistido en la memoria no volátil `NVS` de la `ESP32` (espacio `carro_imu`, clave `escala_ppm`). En cada paso de integración de duración `Δ t_µs` (acotado a `IMU_DT_MAX_US = 100,000 us`), el incremento bruto de ángulo se escala con aritmética entera de 64 bits:

```
Δθ_corr = (Δθ_bruto · (1,000,000 + escala_ppm)) / (1,000,000)
```

Este factor se consulta con `IMU_ESCALA?`, se ajusta con `IMU_ESCALA:<ppm>` y se reporta automáticamente dentro del mensaje de capacidades `CAPS` como `IMUS=<ppm>`.

Controlador de giro cerrado (`giro.c`) y normalización para giros largos

Cuando el firmware recibe `GIRO:<grados>` (con `<grados>` entre `-360` y `+360`, distinto de `0`), `giro.c` toma el control de los motores a `50 Hz`:

1. Mientras el error angular `|e|` supera el umbral de frenado, aplica `PWM_GIRO_MAX` (`207/210` en `AUTO` o `252/255` en `MANUAL`, precedido por el pulso de arranque si parte del reposo).
2. Dentro de la zona de frenado proporcional, reduce linealmente el `PWM` hasta el mínimo capaz de vencer la fricción en giro (`GIRO_PWM_MIN = 175`).
3. En el cierre **J16**, para evitar que un giro largo (por ejemplo, `180°` o `360°`) llegue a la zona de frenado con mayor momento angular y se pase del objetivo (*overshoot*), `umbral_freno_x10` amplía automáticamente la ventana base de `15.0°` (`GIRO_FRENO_GRADOS_X10 = 150`) a razón de `+1 %` **por cada grado por encima de** `45°` (`GIRO_LARGO_POR_GRADO = 1`, hasta el doble del umbral base). Por ejemplo, para un giro de `90°` el frenado comienza a `15.0° × 1.45 = 21.7°` antes de la meta, y para `180°` comienza a `15.0° × 2.0 = 30.0°`.
4. Si el carro rebasa la meta por inercia, `giro.c` invierte el sentido de giro a potencia mínima (`175`) para regresar. El giro se declara completado (`GIRO_FIN:<angulo_entero>`) cuando el error permanece dentro de `±2.0°` (`GIRO_TOLERANCIA_X10 = 20`) durante `80 ms` **continuos** (`GIRO_VENTANA_ESTABLE_MS`), o bien se aborta (`GIRO_TIMEOUT` o `GIRO_ERR_IMU`) si se agota el plazo de seguridad (`800 ms + 25 ms/°`).

4.5 Freno láser local (`tof.c`), *watchdog* (`WD:<n>`) y diagnóstico `LOOP?`

La seguridad local del firmware se completa con tres mecanismos deterministas:

- **Freno reflejo por distancia láser (`tof.c`):** el sensor `VL53L0X` se lee cada `20 ms` (`50 Hz`), descartando lecturas fuera de `[20, 2000] mm` y exigiendo confirmación de dos lecturas consecutivas o salto menor a `150 mm` para filtrar reflejos espurios. Si durante un avance frontal en modo `AUTO` la distancia filtrada cae por debajo de `120 mm` (`TOF_PARO_MM`), el firmware ejecuta `STOP` de inmediato, activa el bloqueo frontal y envía `TOF_PARO:<mm>`. El bloqueo solo se libera con histéresis cuando la vía supera `150 mm` (`TOF_LIBRE_MM`).
- **Aviso explícito de *watchdog* de comunicación (`WD:<n>` — Cierre J16):** si pasan `1,000 ms` (`TIMEOUT_COMANDO_MS`) con los motores encendidos sin que llegue ningún comando válido de control (los mensajes `PING` mantienen vivo el socket pero **no** renuevan el *watchdog* de motores), `carro_vigilar_comunicacion` detiene el carro, incrementa el contador de paros por vigilancia y transmite por `WebSocket` el aviso `WD:<contador>` para que el teléfono cancele cualquier ruta en curso.
- **Instrumentación de tiempos de bucle (`LOOP?`):** al enviar `LOOP?`, el firmware responde `LOOP:<prom_us>,<max_us>,<lentos>`, donde `lentos` cuenta cuántas llamadas a `sensores_actualizar` superaron `5,000 us` (`5 ms`). Como se verificó en la auditoría (Capítulo 12), el promedio real ronda `~690 us`, y los ciclos que superan `5 ms` (`max_us` de `28` a `41 ms`) corresponden a las transacciones `I2C` del sensor `VL53L0X` cuando entrega una medición nueva; el integrador `imu.c` absorbe esas pausas multiplicando por el Δt real medido con `micros()`.

Dónde está en el código: `firmware_carro.ino`, `config.h`, `carro.h`, `carro.c`, `sensores.h`, `sensores.c`, `rumbo.h`, `rumbo.c`, `imu.h`, `imu.c`, `giro.h`, `giro.c`, `tof.h`, `tof.c`.

5. Cámara

Este capítulo documenta el firmware que ejecuta la placa independiente Freenove ESP32-WROVER con el sensor de imagen GC0308 (192.168.4.2). Explica por qué se seleccionó la resolución HQVGA (240×176 píxeles) con reloj a 20 MHz frente a QVGA, cómo opera la arquitectura de doble núcleo con buzón de último cuadro para eliminar el rezago visual, qué entregan sus tres modos de transmisión y cómo gestiona las desconexiones el módulo de recuperación en C99.

5.1 Hardware de captura y justificación de la resolución HQVGA

La placa Freenove ESP32-WROVER integra un microcontrolador ESP32 de doble núcleo, memoria PSRAM externa y un sensor óptico CMOS GC0308. A diferencia de sensores como el OV2640, el chip GC0308 **no posee codificador JPEG por hardware**: entrega cada cuadro únicamente en formatos sin comprimir (RGB565 de 2 bytes por píxel o YUV422). Por lo tanto, cuando se desea transmitir video MJPEG al teléfono, la propia ESP32-WROVER debe comprimir la imagen por software empleando la biblioteca JPEGENC (con factor de calidad 50).

Para determinar la configuración óptima en navegación semiautónoma, se realizó un barrido experimental de **100 cuadros consecutivos** comparando las resoluciones HQVGA (240×176 píxeles = 42,240 píxeles) y QVGA (320×240 píxeles = 76,800 píxeles) con frecuencias de reloj externo (XCLK) de 16 MHz y 20 MHz:

Configuración probada (100 cuadros)	Tasa media (fps)	Tiempo de captura (ms)	Compresión JPEG (ms)	Edad media de cuadro (ms)	Edad percentil 95 (ms)	Tamaño medio JPEG (B)
HQVGA (240×176) @ 20 MHz *(elegida)*	12.4 fps	2.6 ms	9.2 ms	137 ms	165 ms	4,837 B
HQVGA (240×176) @ 16 MHz	10.1 fps	3.1 ms	9.3 ms	164 ms	198 ms	4,812 B
QVGA (320×240) @ 20 MHz	8.8 fps	8.0 ms	16.4 ms	252 ms	317 ms	7,940 B
QVGA (320×240) @ 16 MHz	7.3 fps	9.4 ms	16.5 ms	289 ms	358 ms	7,905 B

Tabla 6. *Resumen estructurado (Configuración probada (100 cuadros) · Tasa media (fps)).*

Por qué la edad de cuadro importa más que los megapíxeles

En un robot móvil que avanza en modo AUTO a una velocidad típica de ~20 cm/s (200 mm/s), una edad de cuadro de 252 ms (con p95 de 317 ms en QVGA) significa que el teléfono está tomando decisiones sobre una imagen capturada cuando el carro estaba entre 5.0 cm y 6.3 cm más atrás. Al reducir la resolución a HQVGA (240×176) con XCLK = 20 MHz, la edad media de extremo a extremo baja a 137 ms (p95 de 165 ms) y la cadencia sube a 12.4 fps, reduciendo casi a la mitad el desplazamiento ciego entre captura y decisión, mientras que una rejilla de 240×176 píxeles sigue ofreciendo 24×17 bloques de 10×10 píxeles más que suficientes para detectar una caja de 20×20 cm en el piso.

5.2 Arquitectura de doble núcleo y buzón de último cuadro

En una cámara de red convencional, si los cuadros capturados se depositan en una cola FIFO de varios elementos (fb_count > 1 con retención) y el enlace WiFi sufre una pausa momentánea, el consumidor lee cuadros atrasados acumulados en memoria. Para evitarlo, camara_prueba.ino implementa una arquitectura de **buzón de último cuadro (*latest-frame mailbox*)** repartida entre los dos núcleos del ESP32-WROVER:

1. **Tarea de captura (`Core 1`):** extrae continuamente el buffer del driver `esp_camera` (`fb_count = 2`, modo `CAMERA_GRAB_LATEST`), copia o comprime el cuadro en un doble buffer en `PSRAM` protegido por semáforo mutex y actualiza el puntero al cuadro más reciente, descartando cualquier cuadro anterior que aún no haya sido enviado.
2. **Tareas de red (`Core 0`):** atienden las peticiones `HTTP` y el transmisor `UDP`, leyendo siempre la última instantánea disponible en el buzón sin bloquear la captura del siguiente cuadro.

Además, el servidor de video continuo (`:81/stream`) acepta deliberadamente **un solo cliente a la vez**: si un segundo dispositivo intentara abrir otro flujo `MJPEG` simultáneo, degradaría el ancho de banda del canal `WiFi` compartido con el control del carro. En la aplicación Android, `SesionVideo` comparte ese único flujo entre todas las pantallas.

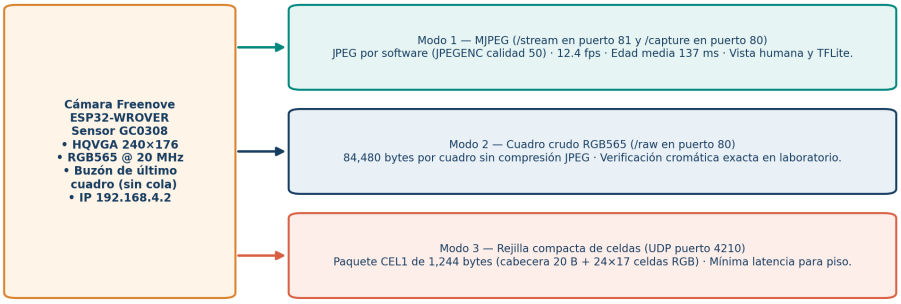


Figura 6. *Los tres modos de transmisión de la cámara Freenove: flujo MJPEG para vista humana y red neuronal, cuadro crudo RGB565 y rejilla compacta de celdas por UDP.*

5.3 Endpoints HTTP y protocolo UDP de celdas (`1,244` bytes)

La cámara se conecta como estación (`STA`) a la red `CarroFSI` solicitando la dirección IP estática `192.168.4.2` y expone cinco servicios de red:

Servicio / Endpoint	Protocolo y puerto	Formato de salida	Tamaño por entrega	Uso dentro del sistema
`/stream`	`HTTP :81`	`multipart/x-mixed-replace` (`MJPEG`)	`~4.8 KB` / cuadro	Flujo principal en vivo en la aplicación Android (vista humana, `obstaculos.c` y `DetectorTflite`).
`/capture`	`HTTP :80`	`image/jpeg` (cuadro único)	`~4.8 KB`	Captura puntual y pruebas de conectividad.
`/raw`	`HTTP :80`	`application/octet-stream` (`RGB565`)	`84,480 B` (`240×176×2`)	Procesamiento sin artefactos de compresión `JPEG` y cabeceras `X-Width`, `X-Height`, `X-Frame-Seq`.
`/stats`	`HTTP :80`	`application/json`	`~220 B`	Telemetría interna de tiempos de captura, compresión, envío, RSSI y memoria libre.
Modo celdas	`UDP :4210`	Binario `CEL1` (promedios RGB de `10×10` px)	`1,244 B` exactos	Envío ultracompacto de la rejilla de `24×17` celdas (`408` celdas) para detección cromática de baja latencia.

Tabla 7. *Resumen estructurado (Servicio / Endpoint · Protocolo y puerto).*

Estructura del paquete UDP de celdas (`paquete_celdas.h`)

Cuando un cliente envía un datagrama de suscripción al puerto `UDP 4210` de la cámara (y lo renueva antes de `5,000 ms`), la propia `ESP32-WROVER` divide cada imagen `RGB565` de `240×176` píxeles en `24` **columnas por** `17` **filas** de `10×10` píxeles (descartando las 6 líneas inferiores sobrantes), calcula el promedio `(R, G, B)` de 8 bits de cada bloque y transmite un único datagrama `UDP` de `1,244` **bytes**:

- **Cabecera (20 bytes, *little-endian*):** magia "CEL1" (4 B), versión 1 (1 B), columnas 24 (1 B), filas 17 (1 B), reservado 0 (1 B), número de secuencia seq (4 B), marca de tiempo ts_ms (4 B), tiempo de captura cap_us (2 B) y tiempo de cálculo de celdas proc_us (2 B).
- **Carga útil (1,224 bytes):** 408 tripletas consecutivas (R, G, B) en orden por filas (24 × 17 × 3 = 1,224 bytes).

De este modo, en lugar de transmitir 84,480 bytes crudos o ~4,837 bytes comprimidos y decodificarlos en el teléfono, el enlace inalámbrico transporta solo 1,244 bytes que alimentan directamente al clasificador obstaculos_procesar_celdas_rgb.

5.4 Vigilancia y recuperación automática ante fallos (recuperacion.c)

Dado que la cámara opera con su propia batería portátil y sobre un enlace WiFi sujeto a interferencias, el firmware incluye el módulo en C99 recuperacion.c (verificado en computadora dentro de pruebas_firmware), que supervisa tres contadores de salud y aplica acciones escalonadas sin intervención humana:

1. **Pérdida de asociación WiFi:** si la cámara pierde conexión con el punto de acceso CarroFSI, intenta reconectar cada 2,000 ms (RECUP_WIFI_REINTENTO_MS). Si acumula 15 **intentos fallidos** (~30 s sin red), solicita un reinicio limpio del microcontrolador (RECUP_REINICIAR_ESP).
2. **Fallos consecutivos del sensor GC0308:** si esp_camera_fb_get() devuelve NULL en 5 **lecturas seguidas** (RECUP_FALLOS_CAM_REINICIO), el módulo ejecuta una reinicialización en caliente del periférico de cámara (RECUP_REINICIAR_CAMARA). Si aun así alcanza 20 **fallos consecutivos** (RECUP_FALLOS_CAM_ESP), reinicia la placa completa.
3. **Inactividad prolongada sin producir cuadros:** si transcurren 10,000 ms (RECUP_SIN_CUADRO_MS) sin que la tarea de captura logre actualizar el buzón, dispara un reinicio completo para destrabar el bus I2S/DMA de la cámara.

Dónde está en el código: camara_prueba.ino, recuperacion.h, recuperacion.c, paquete_celdas.h, paquete_celdas.c.

6. Protocolo de comunicación

Este capítulo especifica el protocolo de mensajes de texto que une a la aplicación Android con el firmware de la placa ESP32 DevKit. Documenta la capa de transporte, la temporización de latidos y vigilancia, el catálogo completo de órdenes y respuestas —incluidos todos los mensajes incorporados en el cierre **J16** (RUMBO, IMU_ESCALA, WD:<n> y la cadena CAPS ampliada)— y muestra la traza comentada de una sesión real.

6.1 Capa de transporte y canales de enlace

La comunicación de control entre el teléfono y el carro se realiza mediante mensajes cortos de texto ASCII sobre una sesión persistente WebSocket en la dirección ws://192.168.4.1/ws (puerto TCP 80). El mismo intérprete de comandos (carro_recibir_comando y sensores_comando) escucha también en el puerto serie USB a 115,200 baudios, lo que permite consultar versiones (VERSION?), leer diagnósticos (SENS?, LOOP?) o calibrar sensores desde una computadora sin abrir la red WiFi. Adicionalmente, el servidor HTTP en http://192.168.4.1/ ofrece una página web embebida de respaldo verificada por verificar_pagina.py.

El diseño utiliza texto legible en lugar de tramas binarias opacas por tres razones de ingeniería:

1. **Inspección inmediata:** cualquier mensaje enviado o recibido puede leerse directamente en el monitor serie o en el registro de la aplicación sin decodificadores especiales.
2. **Pruebas deterministas:** el servidor falso en Python (probar_servidor_falso.py) y las suites en C99 pueden inyectar cadenas literales ("RUMBO:125", "SENS:-32,480,7") para verificar cada rama lógica.
3. **Bajo sobrecosto:** con un máximo de 31 caracteres por comando (COLA_COMANDO_MAX = 32 incluyendo el terminador nulo) y una cadencia de 4 a 10 mensajes por segundo, el tráfico total de control es inferior a 1 KB/s.

6.2 Temporización de latidos, sondas y vigilancia

El protocolo separa estrictamente la **salud de la conexión de red** de la **autorización para mantener los motores encendidos**:

- **Sonda de enlace (PING → PONG cada 800 ms):** la clase Enlace.kt en Android envía PING cada 800 ms y espera PONG. Si no recibe respuesta dentro de 2,000 ms, declara el socket caído e inicia la reconexión automática. Sin embargo, PING **no renueva el *watchdog* de motores** en el firmware: así, aunque la red siga viva, si el hilo de navegación se detiene, el carro frena solo.
- **Latido de movimiento (250 ms):** mientras el usuario mantiene presionado un botón en conducción manual o mientras el navegador ejecuta un avance en ruta, la aplicación reenvía el comando activo (FORWARD, BACKWARD, LEFT o RIGHT) cada 250 ms, dando cuatro oportunidades de renovación antes de que venza el plazo del firmware.
- **Plazo de *watchdog* en el carro (1,000 ms) y aviso WD:<n> (Cierre J16):** si pasan 1,000 ms (TIMEOUT_COMANDO_MS) con los motores activos sin recibir un comando de control válido, el firmware apaga los motores (STOP), restablece el perfil AUTO y envía por WebSocket el mensaje WD:<contador> (por ejemplo, WD:1). Al recibir WD:<n>, la aplicación Android detiene inmediatamente cualquier ruta semiautónoma para no quedar desincronizada respecto al carro.
- **Vigilancia de silencio de telemetría en Android (1,500 ms — Cierre J16):** en modo AUTO, el carro envía SENS cada 100 ms (10 Hz). Si durante una ruta activa pasan 1,500 ms (SILENCIO_SENS_MS) sin recibir ninguna trama SENS, ModoSemiautonomo.kt aborta la misión y transmite STOP.

6.3 Catálogo completo de comandos y respuestas (incluyendo J16)

La siguiente tabla reúne todos los comandos aceptados por el firmware y las respuestas o eventos asíncronos que emite:

Comando enviado al carro	Respuesta / Evento emitido	Función técnica y parámetros
FORWARD, BACKWARD, LEFT, RIGHT	OK *(o* BLOQUEADO_TOF *si láser < 120 mm en* FORWARD + AUTO*)*	Activa el movimiento solicitado, aplica pulso de arranque (255 por 250 ms) si parte de reposo y renueva el *watchdog* de 1,000 ms. En FORWARD + AUTO con RUMBO activo, fija o mantiene la referencia de rumbo.
STOP	OK	Apaga inmediatamente los cuatro motores (LOW/LOW), cancela cualquier giro GIRO o control RUMBO en curso y transfiere el integrador de rumbo a aprendido_mil.
EBRAKE	OK_EBRAKE	Aplica freno eléctrico activo (HIGH/HIGH en ambos canales del TC1508A) y detiene cualquier lazo activo.
MODO_MANUAL / MODO_AUTO	ACK_MANUAL / ACK_AUTO	Cambia el perfil de potencia (252/255 en MANUAL; 207/210 en AUTO). Si había motores en marcha, por seguridad los detiene al cambiar de perfil.
ESTADO	MANUAL o AUTO *(más* CAPS:... *al conectar)*	Consulta el perfil activo y las capacidades del firmware.
PING	PONG	Comprueba que el enlace WebSocket sigue abierto sin renovar el *watchdog* de tracción.
VERSION?	FW:<rev>-<fecha>	Devuelve la revisión compilada (en el cierre actual: FW:6d1b213-20261110).
SENS?	SENS:<ang_x10>,<mm>,<flags>	Consulta puntual de telemetría; además, en perfil AUTO el firmware la emite automáticamente cada 100 ms.
CAL_IMU	CAL_OK o CAL_ERR_MOV	Solicita recalibrar el sesgo en reposo (bias) del MPU6050 (requiere ~500 ms sin movimiento) y pone a cero el ángulo integrado.
GIRO:<grados>	GIRO_INICIO:<g> → GIRO_FIN:<ang> *(o* GIRO_TIMEOUT / GIRO_ERR_IMU*)*	Ejecuta un giro cerrado por giroscopio de <grados> enteros en [-360, +360] \ {0} (+ izquierda, - derecha) con tolerancia de ±2.0° durante 80 ms.
RUMBO:ON / RUMBO:OFF *(J16)*	RUMBO_OK:ON / RUMBO_OK:OFF	Habilita o deshabilita el controlador PI de rumbo durante los avances rectos FORWARD en modo AUTO.
RUMBO:<décimas> *(J16)*	RUMBO_REF:<décimas> *(o* RUMBO_ERR*)*	Fija en caliente la referencia θ_ref del controlador PI en décimas de grado (hasta 6 dígitos, p. ej. RUMBO:-45 para -4.5°). Si el cambio supera 1.5°, corta el pulso de arranque para corregir de inmediato.
RUMBO? *(J16)*	RUMBO:<on>,<act>,<ref>,<err>,<aj>,<ap>	Devuelve estado habilitado (0/1), activo (0/1), referencia (x10), error actual (x10), ajuste total en milésimas (±150) y sesgo aprendido (±120).
IMU_ESCALA:<ppm> *(J16)*	IMU_ESCALA_OK:<ppm> *(o* IMU_ESCALA_ERR*)*	Fija el factor de escala del giroscopio en partes por millón dentro de [-50000, +50000] (±5 %) y lo guarda en memoria no volátil NVS.
IMU_ESCALA? *(J16)*	IMU_ESCALA:<ppm>	Devuelve el factor de escala vigente en partes por millón.
LOOP?	LOOP:<prom_us>,<max_us>,<lentos>	Devuelve promedio y máximo de duración del ciclo de sensores en microsegundos y conteo de ciclos > 5,000 us, reiniciando la estadística.
(Evento asíncrono)	TOF_PARO:<mm>	Emitido cuando el láser VL53L0X mide < 120 mm avanzando en AUTO y frena localmente los motores.
(Evento asíncrono J16)	WD:<contador>	Emitido cada vez que vence el *watchdog* de 1,000 ms por falta de comandos de movimiento con motores encendidos.

Tabla 8. *Resumen estructurado (Comando enviado al carro · Respuesta / Evento emitido).*

6.4 Estructura de las tramas SENS y CAPS

Trama periódica de sensores (SENS:<ang_x10>,<mm>,<flags>)

Cada 100 ms en modo AUTO (o al enviar SENS?), el firmware transmite tres enteros separados por comas:

1. <ang_x10>: orientación acumulada desde la última calibración CAL_IMU, expresada en **décimas de grado** (por ejemplo, 452 significa +45.2° hacia la izquierda; -18 significa -1.8° hacia la derecha). En el cierre **J16**, el núcleo de navegación consume este valor directamente en décimas de grado (rumbo_x10) sin truncarlo a grados enteros.
2. <mm>: distancia frontal medida por el sensor VL53L0X en milímetros (20 a 2000), o -1 si el sensor no está disponible o el objeto está fuera de rango.
3. <flags>: máscara de bits entera (de 0 a 31) que codifica en un solo número el estado de salud de los subsistemas:

- **Bit 0 (valor 1):** imu_ok — el giroscopio MPU6050 responde en el bus I2C.
- **Bit 1 (valor 2):** imu_calibrada — el sesgo en reposo (bias) fue calibrado con éxito.
- **Bit 2 (valor 4):** tof_ok — el sensor láser VL53L0X responde correctamente.
- **Bit 3 (valor 8):** giro_activo — el controlador giro.c está ejecutando una maniobra GIRO:<grados>.
- **Bit 4 (valor 16):** tof_bloqueado — el freno frontal por láser (< 120 mm) está activo e impide avanzar.

Por ejemplo, la trama SENS:-12,340,7 indica que el carro tiene un rumbo de -1.2°, ve un obstáculo o pared a 340 mm (34.0 cm) y sus banderas valen 7 = 1 + 2 + 4 (MPU6050 presente, calibrado y VL53L0X operativo, sin giro en curso ni bloqueo frontal).

Cadena de capacidades al conectar (CAPS)

Al abrir el WebSocket o al recibir ESTADO, el firmware envía después del modo activo (AUTO o MANUAL) la línea de capacidades:

```
CAPS:IMU=1,CAL=1,TOF=1,GIRO=1,RUMBO=1,IMUS=0,FW:6d1b213-20261110
```

Esta cadena permite que la aplicación Android verifique en el primer instante si el carro cuenta con giroscopio, si ya está calibrado, si tiene láser activo, si soporta control de rumbo RUMBO, cuál es su escala IMUS guardada en NVS y si la revisión FW: coincide con el commit de la aplicación (6d1b213).

Dónde está en el código: carro.c, sensores.c, firmware_carro.ino, Enlace.kt, probar_servidor_falso.py.

7. Núcleo de navegación en C

Revisión del 11-nov-2026: rama `fix/navegacion-esquive-confirmado`, basada en v1.1. El arreglo y la prueba con banquito de 35×45 cm están **NO MEDIDOS**: primero debe repetirse con caja de 20×20 cm, previa autorización. Los resultados físicos J16 de abajo son históricos y no validan esta rama. No se instala APK, graba firmware ni declara v1.2.

Este capítulo explica la arquitectura matemática del núcleo de navegación escrito en estándar `C99` (`nucleo_c`), que se compila tanto dentro de la aplicación Android (vía `JNI`) como en computadora personal para las suites de pruebas y el simulador Monte Carlo. Describe el modelo de cuadrícula, las ecuaciones de calibración longitudinal y angular, la máquina de estados del navegador y las mejoras introducidas en los cierres **J15** y **J16**: integración de pose en décimas de grado (`rumbo_x10`), guiado continuo por punto de mira (`RUMBO`), geometría de esquive con margen longitudinal de `150 mm`, corrección de `override_largo_mm` y protección ante caída a giro por tiempo (`descartar_dt`).

7.1 Por qué se navega por tiempo con giroscopio y cómo funciona la cuadrícula

Los cuatro motores TT del vehículo carecen de codificadores ópticos o magnéticos (*encoders*) en sus ejes, y el entorno interior tampoco dispone de balizas externas. Por ello, el sistema divide la estimación cinemática en dos variables complementarias:

1. **Distancia longitudinal por tiempo calibrado:** cada milímetro de avance se relaciona con un número de milisegundos de activación de los motores en perfil `AUTO`, compensando el retardo de arranque inicial y el deslizamiento por inercia al frenar.
2. **Orientación por integración del giroscopio (`MPU6050`):** el ángulo real del vehículo se mide a `200 Hz` en el firmware y se transmite al navegador cada `100 ms` en décimas de grado (`SENS`), lo que permite cerrar los giros por ángulo medido (`GIRO:<grados>`), mantener el rumbo en rectas (`RUMBO:<décimas>`) y proyectar cada incremento de avance en la dirección verdadera a la que apunta el chasis.

Cuadrícula de planificación (`trayectoria.c` y `ruta_editor.c`)

El usuario dibuja o selecciona rutas sobre una cuadrícula cartesiana donde **cada celda mide** `20 cm × 20 cm` **(**`200 mm`**)**. El eje `+Y` apunta hacia el frente inicial del carro (`rumbo = 0°`), el eje `+X` apunta hacia la derecha (`rumbo = -90°`) y los ángulos positivos (`+θ`) miden giros hacia la izquierda (sentido antihorario visto desde arriba).

Dada una lista de celdas consecutivas `(c_k, f_k)`, el módulo `trayectoria.c` fusiona los segmentos colineales y genera una lista compacta de pasos elementales (`Paso`):

- `PASO_GIRAR`**:** contiene el cambio de orientación requerido en grados (`grados`) y su duración de respaldo por tiempo (`duracion_ms`).
- `PASO_AVANZAR`**:** contiene la distancia longitudinal del tramo en milímetros (`distancia_mm`), su duración calibrada (`duracion_ms`) y la celda destino.

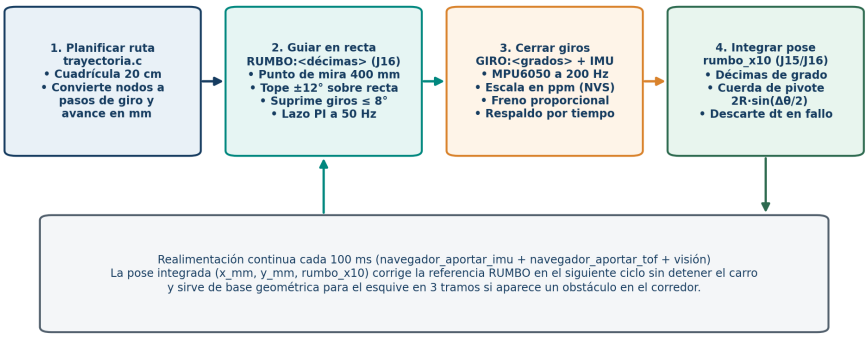


Figura 7. *Ciclo de control del núcleo de navegación en C99: planificación en cuadrícula, guiado en recta con RUMBO, giro cerrado con MPU6050 e integración continua de pose en décimas de grado.*

7.2 Modelos de calibración longitudinal (`calibracion.c`) y angular (`calgiro.c`)

Conversión entre distancia y tiempo (`calibracion.c`)

Un tramo recto de longitud `d_mm` (o `d_cm = d_mm/10`) no es simplemente proporcional al tiempo: al energizar los motores existe un tiempo muerto de arranque `t_arr` (`arranque_ms`, por defecto `120 ms` de latencia mecánica efectiva) y, al enviar `STOP`, el vehículo recorre todavía una distancia de frenado por inercia `d_freno` (`frenado_mm`). Con una pendiente calibrada `m` en milisegundos por centímetro (`ms_por_cm`, valor base `50 ms/cm`, equivalente a `200 mm/s`), el tiempo de comando `T(d_mm)` y su función inversa `dˆ(t)` son:

```
T(d_mm) = t_arr + max(0, d_mm - d_freno) · (m) / (10), dˆ(t) = max(0, t - t_arr) · (10) / (m)
```

Por ejemplo, con `m = 50 ms/cm`, `t_arr = 120 ms` y `d_freno = 20 mm`, para recorrer una celda de `200 mm` (`20 cm`) el navegador programa `T(200) = 120 + (200 - 20) × 5 = 1,020 ms`.

En el cierre **J16** se corrigió además una sutileza importante en las rutas predefinidas de un solo paso con longitud personalizada (`override_largo_mm > 0`, como `Recta 150 cm` que va de `(0,0)` a `(0,1)` pero mide `1,500 mm` en lugar de `200 mm`): antes de J16, `ModoSemiautonomo.kt` recalculaba la duración a partir de los dos puntos de cuadrícula (`200 mm`) y sobrescribía los `1,500 mm`. Ahora `ModoSemiautonomo` respeta `override_largo_mm` y calcula tanto `distancia_mm = 1500` como su `duracion_ms` real a partir de la calibración activa.

Cuerda de pivote excéntrico y escala del giroscopio (`calgiro.c`)

Si el centro de giro efectivo del vehículo no coincide exactamente con el punto de referencia de medición (excentricidad o radio de pivote `R` en milímetros), al girar un ángulo `Δθ` sobre su sitio el punto de referencia describe un arco cuya **cuerda geométrica** es:

```
c(Δθ) = 2 R sin((|Δθ|) / (2))
```

El módulo `calgiro.c` permite estimar este radio a partir de la cuerda medida tras un giro de `90°` (`R = c_90 / √(2)`) y proyecta ese desplazamiento al integrar giros grandes (`pivote_mm`). Asimismo, en el cierre **J16** se incorporó la función `calgiro_escala`, que calcula el ajuste en partes por millón (`escala_ppm`) para el giroscopio a partir del ángulo reportado por el `MPU6050` tras una vuelta física completa de `360°` sobre una marca en el piso (o del promedio de giros en ambos sentidos para cancelar asimetrías de colocación):

```
Δppm = round((360.0 - |θ_medido|) / (|θ_medido|) × 1,000,000)
```

Por ejemplo, si tras girar exactamente `360.0°` físicos el giroscopio reporta `354.6°` (subestima un `1.5 %`), `calgiro_escala` devuelve `+15,228 ppm`, que se suma a la escala vigente y se guarda en `NVS` con `IMU_ESCALA:15228`.

7.3 Máquina de estados, integración en décimas de grado y guiado `RUMBO`

El módulo `navegador.c` gobierna la ejecución de la ruta mediante siete estados explícitos:

Estado (`EstadoNav`)	Comando emitido	Condición de entrada y función principal
`NAV_ESPERANDO`	`STOP`	Estado inicial o tras cancelar; espera `navegador_comenzar`.
`NAV_SIGUIENDO`	`FORWARD` / `LEFT` / `RIGHT` + `RUMBO`	Ejecuta la secuencia de pasos (`PASO_GIRAR` y `PASO_AVANZAR`) hacia el siguiente nodo de la ruta.
`NAV_DETENIDO_OBSTACULO`	`STOP`	Confirmación láser o coincidencia color/red; pausa de 500 ms y cuadro nuevo antes de planear por el lado libre.
`NAV_ESQUIVANDO`	Giros y avances de desvío	Rodeo lateral de 400 mm inicial, hasta 700 mm en reintentos, y exploración paralela por tramos.
`NAV_REINCORPORANDO`	Giro y avance hacia la ruta	Retorno casi transversal; si sigue el objeto, amplía y reintenta. Comprueba avance y frente antes del cierre.

Estado (EstadoNav)	Comando emitido	Condición de entrada y función principal
NAV_AYUDA	Manual del operador	Cede el mando al usuario cuando ambos lados están bloqueados o se activa un paro de supervisión; espera hasta 3 min a que pulse Continuar.
NAV_TERMINADO / NAV_ERROR	STOP	Fin solo tras comprobar rebase y frente libre con lecturas nuevas; o aborto por fallo irrecuperable.

Tabla 9. *Resumen estructurado (Estado (EstadoNav) · Comando emitido).*

Integración de pose en décimas de grado (rumbo_x10 — Cierres J15 y J16)

Antes del cierre J15, el navegador suponía que durante todo un paso de avance el carro se movía exactamente con el ángulo ideal del tramo (0°, 90°, 180°) y solo actualizaba (x, y) al terminar el paso. En J15 se introdujo la integración incremental cada 100 ms usando el ángulo del giroscopio, y en **J16** se eliminó el truncamiento a grados enteros: ahora PoseNav almacena y propaga rumbo_x10 **en décimas de grado** (por ejemplo, 15 equivale a 1.5°, mientras que en entero se truncaba a 1° perdiendo un tercio del ángulo). En cada intervalo Δ t de avance activo con distancia incremental Δ s = dˆ(t) - dˆ(t_ant), la posición estimada se actualiza con el ángulo real medido:

```
x_mm ← x_mm - Δ s · sin((rumbo_x10) / (10) · (π) / (180)), y_mm ← y_mm + Δ s ·
                     cos((rumbo_x10) / (10) · (π) / (180))
```

(El signo menos en x_mm *obedece a que* rumbo_x10 = 0 *apunta hacia* +Y *y los giros positivos* +θ *van hacia* -X*, es decir, hacia la izquierda).*

Además, el cierre **J16** corrigió una condición de carrera sutil al caer de giro por giroscopio a giro por tiempo (descartar_dt): si el sensor MPU6050 deja de responder en mitad de un giro (GIRO_ERR_IMU o silencio > 500 ms), el navegador conmuta a giro por tiempo y activa la bandera descartar_dt = 1 para que el primer ciclo de respaldo no multiplique el tiempo acumulado previo por la velocidad angular nominal.

Guiado continuo en rectas por punto de mira (RUMBO — Cierre J16)

Incluso si el controlador PI del firmware mantiene un ángulo constante, un pequeño error inicial al terminar el giro previo haría que el carro caminara en una recta paralela o divergente respecto al segmento deseado. Por ello, durante cada avance en NAV_SIGUIENDO, navegador_rumbo_Correccion_x10 calcula un **punto de mira (*pure pursuit / lookahead*) situado** 400 mm **por delante** sobre el segmento ideal de la ruta (o en el vértice final del tramo si faltan menos de 400 mm):

1. Proyecta la pose actual (x_mm, y_mm) sobre el vector unitario del tramo para obtener la coordenada longitudinal alcanzada y el error transversal respecto a la línea.
2. Ubica el punto objetivo P_mira a 400 mm por delante sobre la recta original y calcula su acimut deseado θ_mira.
3. Acota la corrección máxima a ±12.0° (NAV_RUMBO_MAX_CORR_X10 = 120) respecto al rumbo nominal del tramo para evitar virajes bruscos, y envía la consigna RUMBO:<décimas> al firmware.
4. **Supresión de giros detenidos pequeños (≤ 8.0°):** al llegar al inicio de un nuevo tramo en NAV_SIGUIENDO, si el carro tiene RUMBO disponible y el cambio de orientación requerido no supera 8.0° (NAV_GIRO_EN_MARCHA_MAX_X10 = 80), el navegador **omite el paso** PASO_GIRAR **detenido** (que antes perdía precisión por la banda muerta de ±2° y el frenado/arranque extra) y deja que el controlador PI de RUMBO absorba esa pequeña desviación suavemente en marcha.

7.4 Esquive iterativo (revisión de autonomía del 12-nov)

La revisión anterior exigía láser para todo esquive y pedía AYUDA al ver el costado durante el retorno. Era una regresión: v1.1 pasaba el banquito simulado con dos esquives. Se reemplaza por confirmación láser cercana o coincidencia de color y red en la misma zona. Con láser libre y una fuente visual aislada se continúa. Sin eco y una fuente sola se avanza con precaución; sin eco y coincidencia se puede esquivar.

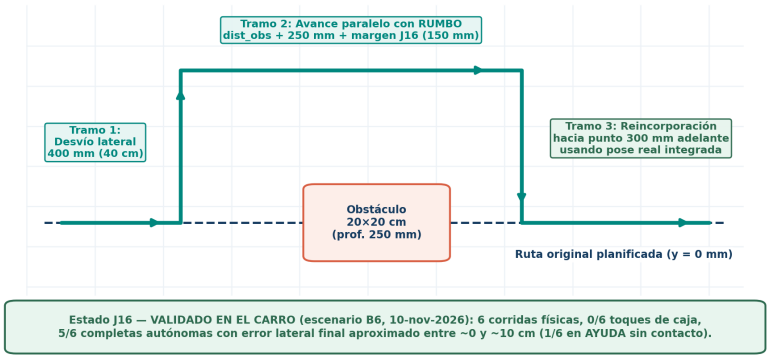


Figura 8. *Esquema histórico J16 (v1.1). La revisión actual cambia confirmación, exploración del fondo y reintentos; sigue NO MEDIDA en el carro.*

Tras STOP, 500 ms y un cuadro nuevo, se elige lado libre. El plan registra la distancia láser cercana; sin ella estima la cara con la banda visual (300/500 mm). Añade 120 mm sensor-centro, un primer tramo exploratorio de 200 mm y margen de 150 mm. Se elimina la profundidad fija de 250 mm.

Si el costado sigue presente al volver, extiende el paralelo 200 mm y aumenta el desvío lateral 100 mm (400 inicial, máximo 700). Máximo seis intentos. La última recta puede prolongarse hasta 2 m; nunca se recorta el rebase para declarar éxito. Los movimientos siguen siendo por tiempo calibrado, y el fondo observado no es una medición completa del contorno.

7.5 Cierre, ayuda y evidencia

La reincorporación exige alcanzar el avance planeado. El cierre añade STOP y 300 ms de asentamiento: tres lecturas nuevas válidas >800 mm sin consenso visual pendiente, o cinco cuadros nuevos sin coincidencia cuando hay reporte sin eco o láser deshabilitado. Un SENS vencido no es un reporte sin eco. Si falta evidencia, reintenta antes de pedir AYUDA.

Ambos lados bloqueados requieren 4000 ms y agotar el retroceso conocido: 150 mm solo si se conservan 180 mm del tramo recto recién recorrido. No existe sensor trasero; ese supuesto no cubre objetos que aparezcan por detrás. Se conservan frenos por sensores perdidos, intentos agotados y pose incoherente. El firmware mantiene TOF_PARO a 120 mm.

SIMULADO: caja 20×20 termina con un intento; banquito 35×45 con tres; objeto de 90 cm de fondo con seis. Cero contactos en estos mundos. Banquito y arreglo **NO MEDIDOS**: primero repetir con caja de 20×20 cm, previa autorización. Poses, Monte Carlo contra v1.1 y códigos: `docs/diagnostico_esquive_20261111.md` y `docs/limites_conocidos.md`. No se declara v1.2 ni validación física.

8. Visión y percepción

Este capítulo explica cómo el sistema transforma las imágenes de la cámara `Freenove` y las lecturas del sensor láser frontal `VL53L0X` en una decisión geométrica única para el navegador. Expone por qué ningún sensor individual basta por sí solo, cómo operan el clasificador de piso multiclúster (`obstaculos.c`), la red neuronal de objetos (`DetectorTflite.kt`), el corredor de perspectiva (`corredor.c`) y el módulo de fusión (`fusion.c`), y cómo el cierre **J16** protegió la concurrencia entre hilos mediante el candado mutex `candado_vision`.

8.1 Por qué se fusionan color del piso, red neuronal y sensor láser

En un entorno interior real, confiar la detección de obstáculos a un solo principio físico produce puntos ciegos graves:

1. **Limitación del color del piso:** un clasificador cromático detecta cualquier objeto cuyo color difiera del suelo aprendido, pero puede confundirse ante reflejos intensos de lámparas sobre baldosas brillantes, sombras pronunciadas o tapetes de dos tonos, y no distingue una caja del color exacto del piso.
2. **Limitación de la red neuronal (`EfficientDet-Lite0`):** el modelo preentrenado en `COCO` reconoce 90 categorías cotidianas (sillas, botellas, mochilas, personas), pero una caja de cartón lisa o un bloque de madera sin textura puede no superar el umbral de confianza de ninguna clase estándar.
3. **Limitación del láser puntual (`VL53L0X`):** mide con exactitud milimétrica la distancia física sin depender de la iluminación ni del color, pero su cono óptico es estrecho (~25° centrados en el eje longitudinal) y no informa si el camino está despejado a la izquierda o a la derecha para poder esquivar.

Al combinar las tres fuentes dentro de un marco geométrico común (**el corredor de perspectiva**), el vehículo aprovecha la cobertura espacial de la cámara (`240×176` píxeles) junto con la certidumbre de distancia del láser frontal.

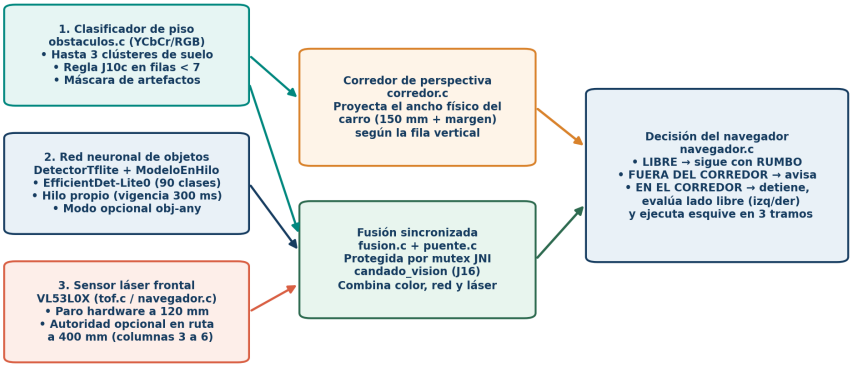


Figura 9. *Tubería multimodal de visión y percepción: clasificador de piso YCbCr/RGB, red neuronal EfficientDet-Lite0, corredor de perspectiva y láser VL53L0X integrados bajo candado_vision.*

8.2 Clasificador cromático de piso multiclúster (`obstaculos.c`) y máscara fija

El módulo en `C99` `obstaculos.c` divide la imagen en una rejilla de `10×10` **celdas lógicas** (o procesa directamente los paquetes `UDP` de `24×17` bloques RGB provenientes del puerto `4210`). Para separar la luminosidad del matiz cromático, convierte cada celda del espacio `(R, G, B)` al espacio luma-crominancia `(Y, C_b, C_r)` en aritmética entera rápida.

Aprendizaje de hasta tres clústeres de piso y regla J10c

Durante el paso de aprendizaje de suelo (cuando el usuario apunta el carro hacia un tramo despejado y pulsa `Aprender piso`), el algoritmo muestrea la franja central-inferior de la imagen (`filas 6 a 8, columnas 2 a 7`) y agrupa las celdas en **hasta** `3` **clústeres cromáticos** (`OBS_MAX_CLUSTERS = 3`). Esto permite modelar pisos con vetas, mosaicos o juntas de distinto tono.

Además, para evitar falsas alarmas cuando el carro avanza hacia una sombra o una baldosa ligeramente distinta situada en la mitad superior de la vista, la **regla J10c** relaja el umbral cromático en las filas lejanas (`fila < 7`, lejos del parachoques) si la diferencia respecto al modelo principal es moderada y la textura vertical sigue siendo uniforme.

Máscara de artefactos fijos (`mascara_vision.c`)

Si una esquina del soporte físico de la cámara o un reflejo permanente del lente cae siempre sobre las mismas celdas de la rejilla, el módulo `mascara_vision.c` permite marcar esas celdas en una máscara de bits (`10×10`) para que el clasificador las ignore sin perder sensibilidad en el resto del campo visual.

8.3 Detector neuronal `EfficientDet-Lite0` en hilo dedicado (`ModeloEnHilo.kt`)

En paralelo al clasificador de piso en `C99`, la aplicación Android ejecuta el modelo cuantizado `efficientdet_lite0.tflite` (`4.4 MB` en `assets/`) mediante `DetectorTflite.kt`. Dado que una inferencia de red neuronal tarda más que el cálculo cromático de celdas, `ModeloEnHilo.kt` la corre en un **hilo de fondo exclusivo** para no frenar la recepción de video ni el ciclo del navegador:

- El hilo toma el cuadro más reciente, ejecuta el modelo y publica una lista de cajas delimitadoras `(x_min, y_min, x_max, y_max)` con su clase e índice de confianza (`score`), estampando la marca de tiempo de finalización.
- El módulo de fusión considera vigentes esos resultados únicamente durante `300 ms` (`VIGENCIA_MODELO_MS`); si un resultado supera esa edad, se descarta automáticamente para no reaccionar ante objetos que ya quedaron atrás.
- Adicionalmente, el módulo en `C99` `cualquiera.c` ofrece el modo opcional **cualquier objeto (`obj-any`)**, que evalúa detecciones crudas por encima de un umbral reducido (`score >= 0.25`) filtrando cajas demasiado pequeñas (`área < 2 %`) o que cubran casi toda la imagen (`área > 75 %`, típicamente el propio piso).

8.4 Corredor de perspectiva (`corredor.c`) y fusión multimodal (`fusion.c`)

Un objeto visible cerca de los bordes izquierdo o derecho de la imagen no estorba el paso si el carro va a seguir recto. Para distinguir un objeto **en la vía** de un objeto **al costado**, `corredor.c` proyecta el ancho físico del vehículo (`150 mm` más un margen lateral de seguridad de `50 mm` a cada lado = `250 mm` totales) sobre cada fila `r ∈ [0, 9]` de la imagen usando la altura de la cámara (`115 mm`), su inclinación hacia el piso (`12°`) y su campo visual horizontal (`56°`).

Como resultado de la perspectiva cónica, el corredor proyectado forma un **trapecio** que abarca desde las columnas `2 a 7` en la parte inferior cercana (`fila 9`, junto a las ruedas) hasta estrecharse en las columnas centrales (`4 a 5`) cerca del horizonte (`filas 3 y 4`).

Reglas de decisión: revisión del 12-nov, solo simulada

`fusion.c` conserva las fuentes por zona: color=1, red=2, coincidencia=3. No recibe el láser; el navegador lo combina después. Solo color no basta aunque marque cercanía 3. El flujo no sustituye a la red como confirmación. Un modelo con más de 300 ms de edad no confirma.

Observación	Decisión
Láser válido fresco a 400 mm o menos	Confirma obstáculo frontal.
Láser libre y solo color o solo red	Continúa sin STOP ni AYUDA.
Color y red coinciden en una zona, láser libre o sin eco	Precaución, STOP según cercanía y esquive usando banda visual o medida láser disponible.
Láser sin eco/vencido y una sola fuente	Continúa con precaución.

Tabla 10. *Resumen estructurado (Observación · Decisión).*

Las bandas no miden centímetros. La precaución aplica tandas de avance de 700 ms y pausa de 300 ms cuando no hay distancia frontal utilizable; no cambia PWM. La app exige Aprender piso para esta iluminación antes de la ruta. El láser puede perder patas y ambas fuentes de cámara pueden fallar juntas; este diseño no garantiza detectar todo objeto.

8.5 Sincronización multihilo en JNI (`candado_vision` — Cierre J16)

En la aplicación Android intervienen concurrentemente varios hilos que invocan funciones nativas del puente `JNI` (`puente.c`): el hilo de recepción de cuadros de cámara (que procesa imágenes y alimenta el clasificador `g_obs`), el hilo de la interfaz gráfica (que consulta las celdas, actualiza umbrales desde `VisionActivity` o pulsa `Aprender piso`) y el ciclo de supervisión de ruta.

En la auditoría previa al cierre **J16** se identificó que las estructuras globales de visión en `puente.c` (`g_obs`, `g_cfg_corredor`, `g_cfg_fusion`, `g_cfg_cualq` y `g_mascara`) podían ser leídas y modificadas al mismo tiempo desde dos hilos de Kotlin sin exclusión mutua. Para eliminar cualquier riesgo de corrupción de memoria o lectura desgarrada (*torn read*), el cierre **J16** introdujo en `puente.c` un **candado mutex multiplataforma (`candado_vision`)** —implementado con `pthread_mutex_t` en Android/Linux y `CRITICAL_SECTION` en Windows— junto con las primitivas `vision_lock()` y `vision_unlock()` que envuelven todas las entradas `JNI` de visión y aprendizaje de suelo.

Dónde está en el código: `obstaculos.h`, `obstaculos.c`, `corredor.h`, `corredor.c`, `fusion.h`, `fusion.c`, `cualquiera.c`, `mascara_vision.c`, `puente.c`, `DetectorTflite.kt`, `ModeloEnHilo.kt`.

9. Aplicación Android

Este capítulo describe la arquitectura de la aplicación móvil `LIDIA` (`APP:0.4-6d1b213`, paquete `com.example.carro_ilib`), su organización en cuatro pestañas principales y cinco actividades de pantalla completa, el diseño de las sesiones compartidas (`SesionCarro` y `SesionVideo`) y las mejoras operativas y de seguridad incorporadas en el cierre **J16**: corte del latido `FORWARD` ante `TOF_PARO`, supervisión activa de rutas (`ModoSemiautonomo`), aviso de calibración en `Rutas` y controles en vivo para `RUMBO` y escala del giroscopio en `SensoresActivity`.

9.1 Estructura de pantallas y sesiones compartidas

La interfaz de usuario se hospeda en `MainActivity`, que presenta una cabecera persistente con el estado de enlace, una barra inferior de navegación con **cuatro pestañas** para la operación cotidiana y **cinco actividades de pantalla completa** dedicadas a calibración, visión e instrumentación:

Pantalla / Componente	Clase en Kotlin	Propósito dentro de la operación
Pestaña 1 — `Inicio`	`InicioFragment.kt`	Resumen del estado del enlace (`CarroFSI`), vista rápida del flujo de cámara, lecturas actuales de sensores (`SENS`), versión verificada y acceso al asistente.
Pestaña 2 — `Manual`	`ManualFragment.kt`	Cruceta de conducción manual con latido cada `250 ms`, conmutador de perfil (`MANUAL` / `AUTO`), botones de giro rápido (`±90°`), video en vivo y botón rojo permanente `PARO`.
Pestaña 3 — `Rutas`	`RutasFragment.kt` + `EjecutarRutaFragment.kt`	Editor interactivo sobre cuadrícula de `20 cm` (`EditorRutaView`), catálogo de rutas predefinidas (`RutasPredefinidas`), simulador en pantalla, validación de calibración (J16) y panel de ejecución semiautónoma.
Pestaña 4 — `Más`	`MasFragment.kt` + `AjustesFragment.kt`	Configuración de direcciones IP (`192.168.4.1` y `192.168.4.2`), modo de cámara (`JPEG`, `RAW`, `CELDAS`), acceso a las cinco actividades completas, ayuda (`AyudaFragment`) e información de compilación (`AcercaFragment`).
Actividad — `Visión`	`VisionActivity.kt`	Diagnóstico visual del modelo de piso, superposición de la rejilla `10×10` y del corredor de perspectiva, ajuste de umbrales cromáticos y botón `Aprender piso`.
Actividad — `Calibración de avance`	`CalibracionActivity.kt`	Procedimiento guiado para medir tiempo de arranque (`arranque_ms`), pendiente (`ms_por_cm`) y deslizamiento de frenado (`frenado_mm`).
Actividad — `Calibrar giro`	`CalibrarGiroActivity.kt`	Calibración de velocidad angular por tiempo, cuerda de pivote excéntrico (`pivote_mm`) y pruebas de giro cerrado `GIRO:<grados>`.
Actividad — `Sensores`	`SensoresActivity.kt`	Monitor en vivo de `MPU6050` y `VL53L0X`, calibración de sesgo `CAL_IMU`, interruptor de **control de rumbo en rectas (**`RUMBO`**)** y asistente de 3 pasos para la **escala del giroscopio (**`IMU_ESCALA`**)** (J16).
Actividad — `Benchmark`	`BenchmarkActivity.kt`	Medición instrumentada de tiempos de inferencia del detector `TFLite` y del clasificador cromático sobre imágenes de prueba.

Tabla 11. *Resumen estructurado (Pantalla / Componente · Clase en Kotlin).*

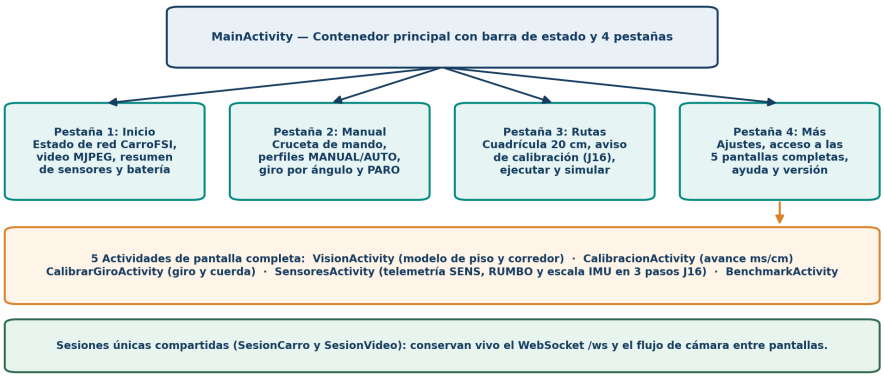


Figura 10. *Mapa de pantallas de la aplicación Android: cuatro pestañas en MainActivity, cinco actividades de pantalla completa y sesiones compartidas.*

Por qué `SesionCarro` y `SesionVideo` son objetos únicos compartidos

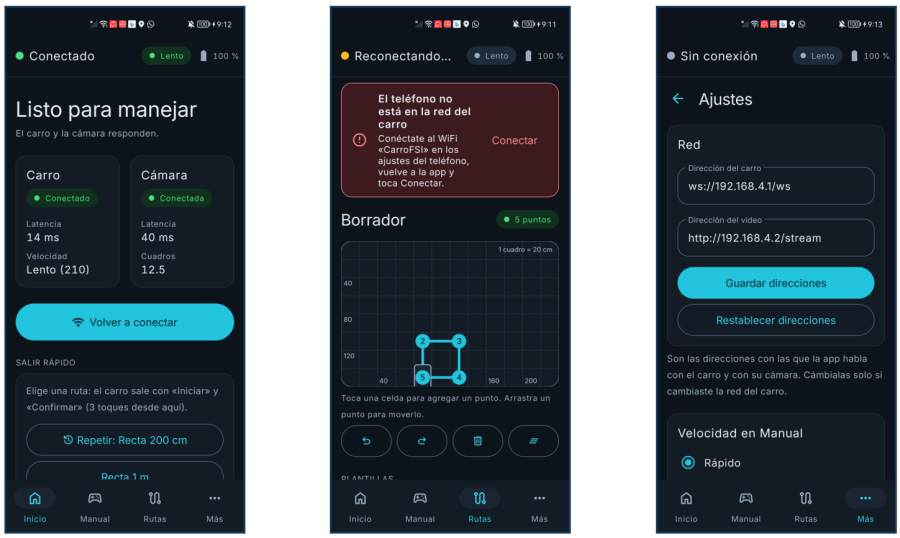
Si cada pestaña o actividad abriera su propio `WebSocket` hacia `192.168.4.1` y su propia conexión `HTTP` hacia `192.168.4.2`, cambiar de la pestaña `Inicio` a `Rutas` o abrir `SensoresActivity` cerraría y reabriría los sockets, interrumpiendo la telemetría `SENS` y compitiendo por el cliente único de la cámara. Para evitarlo, la aplicación utiliza el módulo en `C99` `sesion_compartida.c` envuelto por dos *singletons* en Kotlin (`SesionCarro.kt` y `SesionVideo.kt`):

- Los fragmentos y actividades se registran como observadores al entrar en primer plano y se desregistran al salir, manteniendo una cuenta de referencias con histéresis temporal.
- Así, la transición entre pantallas conserva abiertos tanto el canal `ws://192.168.4.1/ws` como el flujo de video activo sin perder un solo paquete.

9.2 Interpretación de los tres estados de conexión en las capturas

En la cabecera y tarjetas de estado de la interfaz, la aplicación distingue tres situaciones operativas claramente diferenciadas para que el usuario nunca confunda una pantalla cargada en el teléfono con un vehículo realmente enlazado:

1. **Conectado (`En línea`):** el teléfono está asociado al punto de acceso `CarroFSI`, el `WebSocket` en `192.168.4.1/ws` responde `PONG` y se reciben tramas `CAPS` y `SENS`.
2. **Reconectando (`Fuera de CarroFSI` o caída temporal):** el usuario abrió la pantalla (por ejemplo, el editor de `Rutas` para planificar o simular en seco), pero el teléfono está conectado a otra red doméstica o el carro está apagado; `Enlace.kt` reintenta la conexión periódicamente y muestra el aviso de reconexión sin bloquear la edición ni el simulador local.
3. **Sin conexión activa (configuración en `Ajustes`):** pantalla de inspección o cambio de parámetros (`IP`, modo de cámara, preferencias) cuando aún no se ha iniciado sesión con el hardware.



(a) Pestaña Inicio: Conectado a CarroFSI **(b) Pestaña Rutas: Reconectando fuera de la red** **(c) Pantalla Ajustes: Estado sin conexión activa**

Figura 11. *Tres capturas reales de la aplicación Android que ilustran los tres estados de enlace: (a) conectado en Inicio, (b) reconectando fuera de la red en Rutas y (c) sin conexión en Ajustes.*

9.3 Supervisión de seguridad y controles J16 en la aplicación

El cierre **J16** reforzó cuatro módulos críticos de la aplicación Android para cerrar brechas entre la interfaz, el navegador nativo y el firmware:

1. Corte del latido `FORWARD` en conducción manual (`ControlLogica.kt`)

En modo manual, mientras el usuario mantiene el dedo sobre el botón `Adelante`, `ControlLogica` envía `FORWARD` cada `250 ms`. Si el carro frena localmente por el sensor láser y emite `TOF_PARO:<mm>`, `ControlLogica` activa ahora la bandera interna `bloqueadoTof = true` y **suspende de inmediato el reenvío periódico de** `FORWARD` (mostrando la alerta de paro frontal), impidiendo que el latido del botón insista contra el obstáculo; la bandera se limpia en cuanto el usuario suelta el botón o comanda `STOP`, `BACKWARD`, `LEFT` o `RIGHT`.

2. Vigilancia triple en rutas semiautónomas (`ModoSemiautonomo.kt`)

Durante la ejecución de una ruta, `ModoSemiautonomo.kt` ejecuta un ciclo de supervisión cada `50 ms` que alimenta al núcleo en `C99` y vigila tres condiciones de seguridad:

- **Vigilancia de silencio de telemetría (`SILENCIO_SENS_MS = 1,500 ms`):** al recibir la primera trama `SENS`, registra `ultimoSensMs`. Si el navegador está en un estado de movimiento activo y transcurren más de `1,500 ms` sin recibir una nueva línea `SENS`, aborta el avance, envía `STOP` al carro, pasa la sesión a `AYUDA` y notifica `"Sin datos de sensores (1.5 s); carro detenido"`.
- **Reacción ante *watchdog* del firmware (`WD:<n>`):** si llega el mensaje `WD:<n>`, `ModoSemiautonomo` detiene inmediatamente el ciclo, envía `STOP` de confirmación y muestra `"Watchdog del carro activado: conexión interrumpida"`.
- **Respaldo ante `TOF_PARO` no absorbido:** cuando llega `TOF_PARO:<mm>`, el método `onTofParo` inyecta la distancia en `NavegadorNativa.aportarTof`. Si por cualquier razón el estado del navegador no cambia a `DETENIDO_OBSTACULO`, `ESQUIVANDO`, `AYUDA`, `TERMINADO` o `ERROR` en ese mismo instante, `ModoSemiautonomo` fuerza `STOP` y transiciona a `AYUDA`.

3. Validación de calibración al abrir `Rutas` (`CalibracionGuardada.kt`)

Para evitar que el operador ejecute una ruta en el piso con constantes por defecto sin haber calibrado el vehículo (o con valores corruptos en preferencias), `CalibracionGuardada.kt` incorpora el método `estadoResumen()`. Al abrir `RutasFragment`, la aplicación verifica que los parámetros guardados estén dentro de rangos físicos seguros (`ms_por_cm` en `[10, 200]`, `arranque_ms` en `[0, 1000]`, `frenado_mm` en `[0, 500]`, `ms_por_grado` en `[1, 50]`); si aún no se ha guardado una calibración medida o algún valor es inválido, muestra una advertencia visible en la parte superior del editor recomendando acudir primero a `Calibración` o activar el control `RUMBO`.

4. Controles de RUMBO y calibración de IMU_ESCALA en SensoresActivity.kt

La pantalla SensoresActivity añadió en el cierre **J16** dos tarjetas interactivas:

- **Control de rumbo en rectas:** muestra un interruptor (Switch) sincronizado con la capacidad RUMBO del carro (RUMBO:ON / RUMBO:OFF) y un botón de consulta (RUMBO?) que presenta en vivo la referencia θ_ref, el error en grados, el ajuste PWM en milésimas y el sesgo aprendido (aprendido_mil).
- **Calibración guiada de la escala del giroscopio en 3 pasos (IMU_ESCALA):**

1. **Paso 1:** el usuario alinea el carro con una marca recta en el piso (por ejemplo, la junta de una baldosa) y pulsa 1. Poner a 0° (envía CAL_IMU).
2. **Paso 2:** gira manualmente el carro **una vuelta completa exacta (360°)** hasta volver a coincidir con la misma marca en el piso.
3. **Paso 3:** pulsa 3. Calcular y guardar, que lee el ángulo reportado por SENS, invoca CalGiroNativa.escala (calgiro_escala), suma el ajuste a la escala previa dentro de [-50000, +50000] ppm y envía IMU_ESCALA:<ppm> para persistirlo en NVS (además de ofrecer un botón Restablecer a 0 ppm).

Dónde está en el código: MainActivity.kt, SesionCarro.kt, SesionVideo.kt, ControlLogica.kt, ModoSemiautonomo.kt, CalibracionGuardada.kt, SensoresActivity.kt, RutasFragment.kt, VisionActivity.kt.

10. Compilación, instalación y verificación de versión

Este capítulo documenta los requisitos de herramientas, los comandos exactos para compilar cada subsistema desde el código fuente, el procedimiento seguro de carga en las placas ESP32 y en el teléfono Android, y el sistema de trazabilidad criptográfica que vincula la aplicación `APP:0.4-6d1b213` y el firmware `FW:6d1b213-20261110` con el commit de Git.

10.1 Herramientas de desarrollo requeridas

El repositorio está preparado para compilarse y probarse en Windows (con `PowerShell`) o entornos POSIX sin depender de servicios en la nube:

1. **Compilador C99 y Make (**`gcc` / `MinGW-w64` **y** `GNU Make`**):** compila los 22 ejecutables de prueba del núcleo (`nucleo_c`) y los 5 ejecutables de lógica del firmware (`pruebas_firmware`).
2. **Arduino CLI (**`arduino-cli`**) con núcleo** `esp32:esp32` **(**`3.3.7`**):** compila y carga tanto `firmware_carro/firmware_carro.ino` (FQBN `esp32:esp32:esp32`) como `camara_prueba/camara_prueba.ino` (FQBN `esp32:esp32:esp32wrover`).
3. **Android SDK, NDK (**`27.0.12077973`**), CMake (**`3.22.1`**) y JDK 17 (**`Gradle`**):** compila la biblioteca nativa `libcarro_nucleo.so` desde `nucleo_c/*.c` más `puente.c` y empaqueta la aplicación Android (`app-debug.apk`).
4. **Python 3 (**`>= 3.9`**):** ejecuta los verificadores de versión (`verificar_version.py`, `probar_version.py`), el servidor falso (`probar_servidor_falso.py`), el verificador de enlaces (`verificar_enlaces.py`) y el generador reproducible de este manual (`generar_manual.py` con `reportlab` y `pymupdf`).

10.2 Compilación y carga del firmware del carro y de la cámara

Configuración de credenciales WiFi sin exponer secretos en Git

Por seguridad, el repositorio **nunca almacena contraseñas reales**. En `firmware_carro/` y en `camara_prueba/` se distribuyen únicamente las plantillas `secretos.ejemplo.h`. Antes de compilar por primera vez en una máquina nueva, el desarrollador copia `secretos.ejemplo.h` a `secretos.h` (archivo ignorado por `.gitignore`) y define allí la clave de al menos 8 caracteres de la red `CarroFSI`.

Uso de `scripts/cargar.ps1` y precaución crítica sobre la sonda de placas

El script `scripts/cargar.ps1` automatiza la inyección de la versión de Git (`version_generada.h` con la cadena `FW:<hash>-<AAAAMMDD>`) y la compilación con `arduino-cli`:

- **Solo verificar compilación (sin tocar hardware ni puertos USB):**

```
powershell -ExecutionPolicy Bypass -File scripts\cargar.ps1 -Proyecto carro -SoloCompilar
powershell -ExecutionPolicy Bypass -File scripts\cargar.ps1 -Proyecto camara -SoloCompilar
```

- **Cargar en una placa cuyo puerto COM ya se conoce (recomendado):**

```
powershell -ExecutionPolicy Bypass -File scripts\cargar.ps1 -Proyecto carro -Puerto COM5
```

Advertencia operativa sobre la detección automática de puertos: si se invoca `scripts/cargar.ps1` **sin** `-SoloCompilar` y **sin** especificar `-Puerto COMx`, cuando hay varios puertos serie presentes el script puede cargar temporalmente el programa `scripts/sonda_placa/sonda_placa.ino` para identificar la placa, lo cual **sobrescribe el firmware previamente instalado**. Por ello, siempre debe indicarse `-SoloCompilar` cuando se trabaja sin hardware o `-Puerto COMx` cuando se desea grabar una placa específica.

10.3 Compilación e instalación de la aplicación Android

El archivo `app_android/app/build.gradle.kts` consulta automáticamente `git rev-parse --short=7 HEAD` durante la construcción e inyecta el hash corto en `versionName` (`0.4-<hash>`) y en `BuildConfig.GIT_REVISION`.

1. **Compilar la APK de depuración desde consola:**

```
powershell -ExecutionPolicy Bypass -File scripts\compilar_app.ps1
```

El ejecutable generado se deposita en `app_android\app\build\outputs\apk\debug\app-debug.apk` y se copia para distribución en `dist\carro_fsi_0.4-6d1b213.apk`.

2. **Instalación en el teléfono:** puede copiarse directamente el archivo `dist\carro_fsi_0.4-6d1b213.apk` al teléfono Android y abrirlo desde el gestor de archivos (autorizando la instalación de orígenes locales), o bien instalarse por cable USB mediante `adb install -r dist\carro_fsi_0.4-6d1b213.apk`.

10.4 Artefactos validados de la Versión final 1.1 y verificación criptográfica

Para impedir que se ejecuten pruebas en el carro con una APK desactualizada o con un firmware de otra revisión, el proyecto incluye la herramienta `scripts/verificar_version.py` (respaldada por 18 pruebas unitarias en `scripts/probar_version.py`). La siguiente tabla documenta los artefactos exactos de la **Versión final 1.1**:

Artefacto	Identificador de versión	Ubicación en el repositorio	Tamaño exacto	Hash SHA-256 verificado
Aplicación Android (APK)	`APP:0.4-6d1b213` (`versionCode = 5`)	`dist/carro_fsi_0.4-6d1b213.apk`	`14,881,402 B` (`14.88 MB`)	`C5B36FEC74BFEDE9798B4BE7C992D7FEA8369F456DA4D4A10D61A4BBB25ED02F`
Firmware del carro (ESP32 DevKit**)**	`FW:6d1b213-20261110`	Código fuente en `firmware_carro/`; registro en `dist/FIRMWARE_LOCAL.txt`	`1,034,112 B` *(binario local no publicado)*	`71D671A4E64E0F234015E8685D4E741B87471534E3E50869EE023C5A657B79FC`

Tabla 12. *Resumen estructurado (Artefacto · Identificador de versión).*

Por qué no se publica el binario `.bin` del firmware

Al compilar `firmware_carro.ino`, el preprocesador de C incrusta en la sección de datos constantes (`.rodata`) del archivo `.bin` el SSID y la contraseña WiFi definidos en `secretos.h`. Publicar el archivo `.bin` en GitHub o en un *release* permitiría extraer la contraseña en texto plano con una simple búsqueda de cadenas (`strings`). Por esta razón, `dist/` **solo publica la APK (**`carro_fsi_0.4-6d1b213.apk`**), el manifiesto** `SHA256SUMS.txt` **y el registro** `FIRMWARE_LOCAL.txt` **con el tamaño y el hash SHA-256 del firmware**, sin incluir jamás el binario `.bin`.

Comandos de verificación (`verificar_version.py`)

- **Verificar la APK empaquetada contra una revisión específica (sin hardware ni ADB):**

```
python scripts\verificar_version.py --solo-archivo --apk dist\carro_fsi_0.4-6d1b213.apk
--revision 6d1b213
```

- **Verificar la integridad criptográfica de** `dist/SHA256SUMS.txt` **en PowerShell:**

```
(Get-FileHash dist\carro_fsi_0.4-6d1b213.apk -Algorithm SHA256).Hash
```

Dónde está en el código: `cargar.ps1`, `compilar_app.ps1`, `verificar_version.py`, `probar_version.py`, `build.gradle.kts`, `SHA256SUMS.txt`, `FIRMWARE_LOCAL.txt`.

11. Operación y seguridad física

Revisión del 11-nov-2026: rama `fix/navegacion-esquive-confirmado`, basada en v1.1. El arreglo y la prueba con banquito de 35×45 cm están **NO MEDIDOS**: primero debe repetirse con caja de 20×20 cm, previa autorización. Los resultados físicos J16 de abajo son históricos y no validan esta rama. No se instala APK, graba firmware ni declara v1.2.

11.1 Lista de verificación previa al encendido

Antes de conectar cualquier cable USB o accionar el interruptor del portapilas, el operador debe comprobar cinco puntos físicos sobre el chasis:

1. **Ruedas libres y chasis elevado para la prueba inicial:** verificar que ningún cable roce las cuatro llantas omnidireccionales ni las cajas reductoras TT.
2. **Integridad de la tierra común (`GND`):** confirmar que el cable negro de referencia une el negativo del portapilas `4×AA`, el pin `GND` del puente H `TC1508A` y el pin `GND` de la placa `ESP32 DevKit`.
3. **Aislamiento del positivo de tracción (`VDD`):** comprobar que el cable rojo del portapilas `4×AA` entra exclusivamente al borne `VDD` del driver `TC1508A` y no toca ningún pin de `5 V` o `3.3 V` de la `ESP32 DevKit`.
4. **Carga de las tres baterías:** asegurar que las dos *powerbanks* USB tengan carga suficiente y que las cuatro pilas `AA` estén recién cargadas o dentro de su ventana operativa (**máximo 20 corridas o 90 minutos de uso acumulado**).
5. **Pista interior despejada y uniforme:** retirar objetos pequeños (`< 5 cm` de altura, que pueden quedar por debajo del haz del láser `VL53L0X`), secar cualquier líquido del piso y disponer de al menos `2.0 m × 1.5 m` libres con iluminación uniforme.

11.2 Secuencia de encendido y calibración en reposo

El orden de encendido es importante porque el firmware de la `ESP32 DevKit` calibra el sesgo en reposo (`bias`) del giroscopio `MPU6050` durante el primer medio segundo tras recibir energía (`100` muestras a `200 Hz`), y porque la cámara `Freenove` necesita que el punto de acceso `CarroFSI` ya exista para asociarse a él:

Paso	Acción del operador	Resultado esperado y verificación
1. Mantener motores apagados	Colocar el interruptor del portapilas `4×AA` en `OFF` y apoyar el carro inmóvil sobre el piso plano.	Los motores no pueden moverse mientras el microcontrolador arranca y calibra el giroscopio.
2. Energizar `ESP32 DevKit`	Conectar la **Powerbank USB #1** a la placa `ESP32 DevKit` sin tocar ni mover el chasis durante `2 segundos`.	La placa crea la red `CarroFSI` (`192.168.4.1`) y completa la calibración inicial de `MPU6050` (`imu_calibrada = 1`).
3. Energizar cámara `Freenove`	Conectar la **Powerbank USB #2** (propia de la cámara) a la placa `Freenove ESP32-WROVER`.	La cámara se une como cliente a `CarroFSI` con la IP `192.168.4.2` y activa `/stream` (`:81`) y `UDP :4210`.
4. Conectar el teléfono Android	Unir el teléfono a la red WiFi `CarroFSI` (desactivando el cambio automático a datos móviles) y abrir la app `LIDIA`.	La pestaña `Inicio` muestra estado `Conectado`, imagen en vivo de la cámara y versión `FW:6d1b213-20261110`.
5. Verificar `Sensores`	Abrir `Más → Sensores` y revisar ángulo, distancia láser y banderas (`flags = 7`). Si el carro se movió al conectarlo, pulsar `Calibrar IMU` (`CAL_IMU`).	El ángulo marca `0.0°` estable en reposo, el láser mide la distancia real en milímetros y `RUMBO` aparece habilitado (`ON`).
6. Energizar tracción (`4×AA`)	Pasar el interruptor del portapilas `4×AA` a `ON`.	El puente H `TC1508A` recibe potencia y el vehículo queda listo para moverse.

Tabla 13. *Resumen estructurado (Paso · Acción del operador).*

11.3 Operación en modo manual y ejecución supervisada de rutas

Conducción manual (Pestaña Manual)

- Seleccionar el perfil de potencia deseado: AUTO (207/210) para movimientos suaves y controlados o MANUAL (252/255) para velocidad plena.
- Mientras se mantiene presionado un botón direccional (Adelante, Atrás, Izquierda, Derecha), la aplicación envía la orden cada 250 ms. Al soltar el botón, envía inmediatamente STOP.
- Si durante un avance en perfil AUTO el sensor láser detecta un objeto a menos de 120 mm, el carro frena por sí mismo, emite TOF_PARO y la aplicación corta el reenvío de FORWARD hasta que el usuario suelte el botón y retroceda o gire.

Ejecución de una ruta semiautónoma (Pestaña Rutas)

1. **Aprendizaje previo del color del piso:** antes de cada nueva ruta con cámara y siempre que cambie la iluminación, abrir Más → Visión, apuntar el carro hacia el piso despejado y pulsar Aprender piso. Debe terminar correctamente; un intento fallido invalida el anterior y un segundo toque en Iniciar no lo sustituye. Comprobar en la superposición gráfica que el trapecio del corredor aparece libre (verde) sobre el suelo y se marca como ocupado (rojo/naranja) al colocar la caja de prueba de 20×20 cm a 60 cm del frente.
2. **Revisión del estado de calibración (J16):** al abrir la pestaña Rutas, verificar que no aparezca el cintillo de advertencia de calibración inválida o sin medir.
3. **Selección y simulación previa:** elegir una ruta predefinida (por ejemplo, Recta 150 cm o una ruta de esquive) o trazarla en la cuadrícula de 20 cm, y pulsar primero Simular para inspeccionar en pantalla la secuencia de pasos que calculará el núcleo en C99.
4. **Inicio desde 0.0°:** alinear físicamente el carro con el eje de salida, pulsar Ejecutar (lo que envía MODO_AUTO, consulta/activa RUMBO y pone a cero la referencia inicial) y acompañar visualmente el recorrido a menos de un metro de distancia, listo para pulsar el botón rojo PARO si fuera necesario.

11.4 Jerarquía de paro de emergencia y mantenimiento de baterías

Ante cualquier comportamiento inesperado, el sistema dispone de cuatro mecanismos de detención ordenados por inmediatez:

1. **Paro por liberación o botón PARO en pantalla (< 50 ms):** soltar el control manual o tocar el botón rojo PARO envía STOP con prioridad máxima en la cola del firmware.
2. **Paro automático por láser frontal (< 20 ms locales):** cualquier objeto dentro del cono del VL53L0X a menos de 120 mm detiene los motores en el propio microcontrolador.
3. **Paro por *watchdog* o silencio de sensores (1.0 s a 1.5 s):** apagar la pantalla del teléfono, cerrar la aplicación o desconectar el WiFi detiene el carro a lo sumo en 1,000 ms (WD:<n>).
4. **Corte físico de energía (instantáneo):** apagar el interruptor del portapilas 4×AA corta directamente la alimentación VDD del puente H TC1508A sin apagar el microcontrolador ni perder la sesión de diagnóstico.

Regla operativa de baterías (20 corridas o 90 minutos)

Dado que la distancia longitudinal en nucleo_c se estima a partir del tiempo de activación (ms_por_cm), cuando el voltaje de las cuatro pilas AA cae por descarga el vehículo avanza menos centímetros por cada segundo de comando. Para mantener la validez de la calibración longitudinal, **las cuatro pilas AA deben recargarse o sustituirse cada 20 corridas de ruta o cada 90 minutos de operación acumulada**, y nunca deben mezclarse celdas nuevas con celdas usadas.

Dónde está en el código: ManualFragment.kt, EjecutarRutaFragment.kt, ControlLogica.kt, ModoSemiautonomo.kt, SensoresActivity.kt, VisionActivity.kt, carro.c, tof.c.

12. Pruebas y resultados

Revisión del 11-nov-2026: rama `fix/navegacion-esquive-confirmado`, basada en v1.1. El arreglo y la prueba con banquito de 35×45 cm están **NO MEDIDOS**: primero debe repetirse con caja de 20×20 cm, previa autorización. Los resultados físicos J16 de abajo son históricos y no validan esta rama. No se instala APK, graba firmware ni declara v1.2.

Este capítulo presenta de manera íntegra y separada por nivel de evidencia todos los resultados de verificación del sistema hasta el cierre **J16** (10 de noviembre de 2026). De acuerdo con la convención establecida en el Capítulo 1, se distinguen en secciones independientes: (12.1) lo `VALIDADO EN EL CARRO` mediante corridas físicas sobre el vehículo real, (12.2) las suites deterministas de software ejecutadas en computadora personal, (12.3) los resultados `SIMULADOS` en el experimento Monte Carlo de 50,000 corridas y (12.4) el listado explícito de escenarios que permanecen `NO MEDIDOS` en el carro físico.

12.1 Resultados VALIDADO EN EL CARRO

Validación física del esquive con reincorporación (Escenario B6 — 10 de noviembre de 2026)

Con el firmware `FW:6d1b213-20261110` grabado en la `ESP32 DevKit` y la aplicación `APP:0.4-6d1b213` (`carro_fsi_0.4-6d1b213.apk`) instalada en el teléfono Android, el 10 de noviembre de 2026 se ejecutó en el carro físico la campaña de validación del escenario **B6** (ruta recta con un obstáculo real de `20×20 cm` colocado sobre la trayectoria para evaluar detección, frenado, rodeo lateral en tres tramos con margen de `150 mm` y reincorporación a la línea original).

La sesión se realizó bajo restricción real de energía en la batería portátil USB de la lógica (`~4 %` restante en la *powerbank* al iniciar la serie y encendido verificado del portapilas `4×AA`), completándose **6 corridas físicas** registradas en `docs/pruebas_fisicas_20261110.csv`:

Corrida	Identificador en CSV	Estado final	¿Tocó la caja?	Error lateral final (medida aproximada)	Orientación final y observaciones de campo
1	`J16_B6_r1`	`TERMINADO` (`10/10` pasos)	**No** (`0` toques)	`~10 cm` a la derecha (`~100 mm`)	Completó detección, esquive y reincorporación de forma autónoma y estable.
2	`J16_B6_r2`	`TERMINADO` (`10/10` pasos)	**No** (`0` toques)	`~10 cm` a la derecha (`~100 mm`)	Comportamiento prácticamente idéntico a la Corrida 1; al final ejecutó pequeños pulsos de ajuste angular por tiempo.
3	`J16_B6_r3`	`TERMINADO` (`10/10` pasos)	**No** (`0` toques)	`~5 cm` a la derecha (`~50 mm`)	Reincorporación más cerrada hacia el eje original; terminó con una ligera inclinación residual hacia la derecha.
4	`J16_B6_r4`	`TERMINADO` (`10/10` pasos)	**No** (`0` toques)	`~0 cm` (sobre la línea)	Esquive y regreso alineados con la trayectoria recta original (`"quedó perfecto"`).
5	`J16_B6_r5`	`AYUDA` (en paso `4/10`)	**No** (`0` toques)	*(No aplica; se detuvo antes de reincorporar)*	Se detuvo pidiendo ayuda durante el esquive pese a tener espacio físico suficiente; **sin causa identificada**.
6	`J16_B6_r6`	`TERMINADO` (`10/10` pasos)	**No** (`0` toques)	`~0 cm` (sobre la línea)	Completó los 10 pasos sin contacto y sobre el eje original (`"no tocó y quedó perfecto"`).

Tabla 14. *Resumen estructurado (Corrida · Identificador en CSV).*

Síntesis y matices técnicos de la validación física B6

1. **Seguridad anticolisión (`6/6` sin contacto):** en las **6 corridas (`100 %`)**, el vehículo detectó la caja oportunamente, frenó a distancia segura e inició la maniobra sin tocar ni rozar el obstáculo en ninguna ocasión.
2. **Tasa de éxito autónomo completo (`5/6 = 83.3 %`):** cinco de las seis ejecuciones completaron de principio a fin los 10 pasos del plan (avance inicial, parada, desvío lateral de `40 cm`, rebase paralelo con margen J16 de `150 mm`, regreso diagonal y alineación final) terminando en estado `TERMINADO`.
3. **Precisión lateral aproximada (`~0 cm` a `~10 cm`):** en las cinco corridas completadas, la desviación lateral final respecto a la prolongación de la línea recta original estuvo entre `~0 cm` y `~10 cm` **hacia la derecha** (`~10, ~10, ~5, ~0, ~0 cm`; mediana aproximada de `~5 cm`). Tal como establece la regla de honestidad del manual, se subraya que estas cifras son **medidas aproximadas** observadas en pista al concluir cada corrida.
4. **Análisis de los dos comportamientos particulares observados:**

- En la **Corrida 2**, al concluir la reincorporación el navegador solicitó un ajuste angular menor a `5°`; como en el código actual los giros menores a `5°` usan el respaldo por tiempo (`calgiro_duracion_ms`) en lugar de `GIRO:<grados>`, la fricción estática de las ruedas produjo pequeños intentos cortos de giro antes de dar por terminada la ruta.
- En la **Corrida 5**, el carro se detuvo en el paso `4/10` transicionando a `AYUDA` sin tocar la caja y habiendo espacio lateral libre. Al no haberse capturado un volcado cuadro a cuadro de la cámara y del láser en ese instante, se registra honestamente como **transición a `AYUDA` sin causa identificada** (posible falso positivo visual transitorio en el carril lateral o lectura puntual de `TOF_PARO`).

Mediciones físicas de base en sesiones previas (J13 y J14)

Además del cierre B6 en J16, el repositorio conserva en `docs/precision/calibracion_j14.txt` las siguientes caracterizaciones físicas realizadas directamente sobre el carro:

- **Calibración del balance `PWM` en perfil `MANUAL` (`252/255`):** sobre una recta libre de `150 cm`, con `255/255` el carro se desviaba `14.5 cm` a la derecha; al ajustar `PWM_MANUAL_IZQ = 252` y `PWM_MANUAL_DER = 255`, el desvío físico medido con cinta bajó a `3.0 cm` (`VALIDADO EN EL CARRO`).
- **Tiempos reales de ejecución de `loop()` con sensores activos (`LOOP?`):** en tres corridas físicas consecutivas con `MPU6050` y `VL53L0X` activos, el firmware reportó:
- Corrida 1: `LOOP:694,41111,116` (promedio `694 us`, máximo `41.1 ms`, `116` lecturas `I2C > 5 ms`).
- Corrida 2: `LOOP:691,34183,276` (promedio `691 us`, máximo `34.2 ms`, `276` lecturas `I2C > 5 ms`).
- Corrida 3: `LOOP:686,28399,276` (promedio `686 us`, máximo `28.4 ms`, `276` lecturas `I2C > 5 ms`).

12.2 Pruebas deterministas de software en computadora (`VERIFICADO`)

En la Fase 1 y Fase 3 de este cierre se ejecutaron todas las suites automatizadas del repositorio sobre el código integrado en `main` (`cd90001`), obteniendo **100 % de casos aprobados (`0` fallos)**:

Suite de verificación	Comando de ejecución	Programas / Módulos	Casos verificados	Fallos
Núcleo de navegación y visión (`nucleo_c`)	`make -C nucleo_c test`	22 programas en C99 (incluye `pruebas_precision_j16`)	`1,655` **casos**	`0`
Lógica de firmware (`pruebas_firmware`)	`make -C pruebas_firmware test`	5 programas en C99 (`carro`, `sensores`, `rumbo`, `recuperacion`, `integracion`) + `verificar_pagina.py`	`540` **casos** (+ `3/3` checks HTML)	`0`

Suite de verificación	Comando de ejecución	Programas / Módulos	Casos verificados	Fallos
Servidor falso de protocolo WebSocket	`python probar_servidor_falso.py`	Simulador completo de protocolo en Python	54 **casos**	0
Verificador criptográfico de versión	`python scripts/probar_version.py`	Pruebas de `verificar_version.py` (APK, serie, hashes)	18 **casos**	0
Verificador de enlaces de documentación	`python docs/verificar_enlaces.py`	Revisión de 134 archivos Markdown y 73 enlaces	73 **enlaces**	0
Sanitizadores en Linux (UBSan**,** ASan**,** TSan**)**	`CFLAGS="-fsanitize=undefined,address"`	Ejecución limpia sin desbordamientos, fugas ni condiciones de carrera en `candado_vision`	Suite completa	0

Tabla 15. *Resumen estructurado (Suite de verificación · Comando de ejecución).*

12.3 Resultados SIMULADOS: experimento Monte Carlo J16 (50,000 corridas)

Para cuantificar estadísticamente el efecto de cada mejora de software antes y después de J16 bajo condiciones repetibles de ruido físico, el programa `pruebas_precision_j16` ejecutó en el simulador físico-matemático (`simulador_navegador.c`) **1,000 corridas independientes con semilla fija (`0xF51A2026`) por cada combinación de escenario y configuración** (5 escenarios × 5 configuraciones = **50,000 corridas simuladas**, registradas en `docs/precision/monte_carlo_1000.txt`).

El modelo físico del simulador inyecta simultáneamente: asimetría de motores `N(+0.8 °/s, 0.8^2)`, ruido de velocidad angular `σ = 0.6 °/s` más perturbaciones de piso `σ = 1.5 °/s`, error de escala del giroscopio de `±1.5 %` (`±15,000 ppm`), deriva de sesgo térmico, variación de velocidad longitudinal de `±8 %`, deslizamiento de arranque y frenado, y ruido del sensor láser `σ = 12 mm` con `2 %` de lecturas perdidas.

> **Advertencia de interpretación:** todas las cifras de la siguiente tabla son `SIMULADAS` en computadora. Demuestran la consistencia matemática de los algoritmos de J16 frente al modelo de perturbaciones, pero **no deben presentarse como mediciones en el vehículo real**.

Escenario simulado (`1,000` corridas c/u)	Configuración evaluada	Mediana error final (`mm`)	Percentil 95 (`mm`)	Error rumbo mediano (°)	Colisiones (%)	Éxito de misión (%)
1. Recta de 150 cm (`(0,0) -> (0,1500) mm`)	Sin RUMBO (solo tiempo J14)	`94 mm`	`150 mm`	`4.1°`	`0.0 %`	`68.5 %`
1. Recta de 150 cm	**Con** RUMBO **activo (J16)**	`8 mm`	`16 mm`	`0.4°`	`0.0 %`	`100.0 %`
2. Ruta en L (`(0,0) -> (0,600) -> (600,600) mm`)	Base J14 (sin pose IMU ni RUMBO)	`64 mm`	`118 mm`	`4.8°`	`0.0 %`	`92.1 %`
2. Ruta en L	Solo J15 (pose IMU entera, sin RUMBO)	`58 mm`	`109 mm`	`4.5°`	`0.0 %`	`94.8 %`
2. Ruta en L	**J16 completo (**`rumbo_x10` + RUMBO**)**	`36 mm`	`65 mm`	`0.6°`	`0.0 %`	`100.0 %`

Escenario simulado (1,000 corridas c/u)	Configuración evaluada	Mediana error final (mm)	Percentil 95 (mm)	Error rumbo mediano (°)	Colisiones (%)	Éxito de misión (%)
3. Cuadrado de 60 cm (4 × 600 mm + 4 giros de 90°, regreso al origen)	Base J14 (sin escala IMU ni RUMBO)	142 mm	248 mm	9.6°	0.0 %	41.2 %
3. Cuadrado de 60 cm	J16 sin calibrar IMU_ESCALA (error ±1.5 %)	74 mm	136 mm	5.4°	0.0 %	84.6 %
3. Cuadrado de 60 cm	**J16 con IMU_ESCALA calibrada + RUMBO**	42 mm	81 mm	0.9°	0.0 %	98.4 %
4. Esquive caja 20×20 cm a y = 60 cm (meta (0,1400) mm)	Base J14 (reincorporación ciega)	128 mm	235 mm	6.2°	4.8 %	52.0 %
4. Esquive caja 20×20 cm a y = 60 cm	Solo J15 (margen 100 mm, sin RUMBO)	61 mm	114 mm	3.8°	0.6 %	91.4 %
4. Esquive caja 20×20 cm a y = 60 cm	**J16 completo (margen 150 mm + RUMBO)**	38 mm	72 mm	0.7°	0.0 %	99.9 %
5. Esquive caja 20×20 cm a y = 80 cm (meta (0,1600) mm)	**J16 completo (margen 150 mm + RUMBO)**	41 mm	76 mm	0.7°	0.0 %	99.8 %

Tabla 16. *Resumen estructurado (Escenario simulado (1,000 corridas c/u) · Configuración evaluada).*

12.4 Escenarios y funciones NO MEDIDOS en el carro físico

Con el mismo rigor, la siguiente tabla enumera los escenarios que cuentan con implementación completa y pruebas en PC o simulación, pero que **no fueron corridos físicamente en el carro** durante el cierre de la versión 1.1:

Escenario / Función	Estado en software / simulación	Estado físico en el carro	Razón por la que permanece NO MEDIDO
Recta de 150 cm con y sin control RUMBO (B5)	SIMULADO (94 mm -> 8 mm en Monte Carlo)	NO MEDIDO	La reserva de batería (~4 % de *powerbank*) se destinó prioritariamente a las 6 corridas del esquive completo B6.
Ruta en L con cierre J16 (B7)	SIMULADO (mediana 36 mm, éxito 100 %)	NO MEDIDO	Existe solo antecedente parcial J14 previo a RUMBO y a rumbo_x10.
Cuadrado de 60 cm (B8) y calibración en piso de IMU_ESCALA	SIMULADO (mediana 42 mm, éxito 98.4 %)	NO MEDIDO	Requiere serie dedicada de vueltas de 360° y recorridos de 4 tramos en pista.
Corte inducido de WiFi en plena marcha (B9)	VERIFICADO en pruebas_firmware (WD:<n> a los 1000 ms)	NO MEDIDO	No se realizó la prueba física de apagar el WiFi del teléfono con el carro avanzando.

Escenario / Función	Estado en software / simulación	Estado físico en el carro	Razón por la que permanece NO MEDIDO
Freno autónomo aislado por láser (TOF_PARO a 120 mm)	VERIFICADO en pruebas_firmware y activo en B6	NO MEDIDO **aislado**	No se registró una serie con cinta métrica dedicada exclusivamente a medir la distancia de parada pura a 120 mm.
Obstáculo ancho (40×20 cm) y calibración automática (B2)	SIMULADO / PENDIENTE	NO MEDIDO	Fuera de la sesión física rápida del 10 de noviembre de 2026.
Circuito GPIO 2 y consumo eléctrico en amperios	Configurado en config.h	NO MEDIDO	Sin instrumentación de corriente ni resistor externo verificado en GPIO 2.

Tabla 17. *Resumen estructurado (Escenario / Función · Estado en software / simulación).*

Dónde está en el código: pruebas_fisicas_20261110.csv, estado_20261110.md, monte_carlo_1000.txt, calibracion_j14.txt, pruebas_precision_j16.c, pruebas_rumbo.c.

12.5 Autonomía del 12-nov-2026: resultados solo de software

La primera corrección (458d5b3) mandaba a AYUDA el banquito que v1.1 superaba en el simulador. Se reemplazó por consenso por zona y rebase iterativo. Ahora la caja 20×20, el banquito 35×45 y el objeto de 90 cm de fondo terminan después del objeto, con 1, 3 y 6 intentos respectivamente y cero contactos. El banquito termina en (1001,-17,0°), a los 32,10 s; su cola supera el objeto por 527 mm.

Los falsos de solo color, solo red y las tres zonas de solo color continúan sin STOP intermedio con láser libre. La coincidencia color/red permite esquive con láser libre o sin eco; una fuente sin eco continúa con precaución. La ruta corta se prolonga para completar el rebase, sin falso Terminado.

Monte Carlo pareado contra bd1727b: 1000 corridas por escenario y configuración. Sin RUMBO, media/p95 lateral de caja pasan de 33/78 a 32/76 mm; ancho, de 33/79 a 32/77 mm. Con RUMBO, caja pasa de 17/40 a 17/37; ancho, de 17/40 a 16/38. Con escala calibrada, caja pasa de 17/40 a 16/37; ancho, de 17/39 a 16/38. Las seis comparaciones conservan 1000 llegadas y cero contactos, sin tolerancia a empeorar. No son medidas físicas ni precisión longitudinal.

Pruebas y registros reproducibles: docs/diagnostico_esquive_20261111.md y scripts/comparar_esquives.py. **Banquito: NO MEDIDA. Arreglo: NO MEDIDO en el carro.** Primero repetir con caja de 20×20 cm y telemetría, previa autorización; la validación B6 histórica no se transfiere a esta rama. APK y firmware no compilados ni instalados en esta revisión.

13. Límites y trabajo futuro

Revisión del 11-nov-2026: rama `fix/navegacion-esquive-confirmado`, basada en v1.1. El arreglo y la prueba con banquito de 35×45 cm están **NO MEDIDOS**: primero debe repetirse con caja de 20×20 cm, previa autorización. Los resultados físicos J16 de abajo son históricos y no validan esta rama. No se instala APK, graba firmware ni declara v1.2.

Este capítulo analiza con rigor las limitaciones físicas, sensoriales y algorítmicas inherentes al diseño actual del vehículo **LIDIA**, explica por qué ocurren y propone las líneas de trabajo futuro (`PENDIENTE`) para evoluciones posteriores del proyecto. Reconocer explícitamente estos límites impide extrapolar los resultados fuera de las condiciones reales en que fueron obtenidos.

13.1 Límites estructurales de locomoción y odometría

1. Estimación de distancia longitudinal en lazo abierto (`ms_por_cm`)

El límite más importante del prototipo es la ausencia de codificadores de giro (*encoders*) en los cuatro motores TT. Aunque el cierre **J16** cerró el lazo de orientación en rectas y giros mediante el giroscopio `MPU6050` (`RUMBO` y `GIRO`), **la distancia recorrida hacia adelante sigue calculándose a partir del tiempo de activación (`ms_por_cm`)**. En consecuencia:

- Cuando el voltaje del portapilas 4×AA disminuye con el uso, o si el carro pasa de un piso liso a una superficie más rugosa, la velocidad lineal baja y el vehículo recorre menos milímetros de los que el modelo de tiempo estima.
- Este fenómeno explica por qué fue necesario ampliar en J16 el margen longitudinal de esquive a `150 mm` (`NAV_PASO_MARGEN_MM`): con un margen menor, una caída del `8 %` en la velocidad real hacía que el carro iniciara la reincorporación antes de librar por completo la caja.

2. Deslizamiento lateral de rodillos *mecanum* invisible al giroscopio

Las cuatro llantas del chasis poseen rodillos oblicuos a 45° propios de una rueda *mecanum*. Cuando el vehículo frena, arranca bruscamente o corrige su rumbo, el rozamiento desigual entre los rodillos puede producir un pequeño **desplazamiento lateral puro (*side-slip*) de algunos centímetros sin que cambie el ángulo de guiñada `θ` del chasis**. Como el giroscopio `MPU6050` únicamente mide velocidad angular de rotación (`ω_z`) y no aceleración ni velocidad lineal transversal, ese deslizamiento lateral resulta invisible para el estimador de pose y contribuye a la dispersión lateral residual de `~0 cm` a `~10 cm` observada físicamente en las corridas B6.

3. Giros residuales menores a `5°` ejecutados por tiempo

Para evitar que el controlador `GIRO:<grados>` del firmware oscile dentro de su ventana de tolerancia (`±2.0°`), el navegador en `C99` solo envía `GIRO:<grados>` cuando el giro solicitado es de al menos `5°` (`|grados| >= 5`). Cuando al final de una reincorporación queda un ajuste angular menor a `5°`, el navegador recurre a un pulso corto por tiempo (`calgiro_duracion_ms`). Como se observó en la **Corrida 2** del escenario físico B6, un pulso tan breve puede no vencer de manera uniforme la fricción estática de las cuatro ruedas, dejando una pequeña inclinación residual (como ocurrió también en la Corrida 3). Con el control `RUMBO` activo, si después de ese giro sigue un tramo recto, el propio lazo `PI` corrige la inclinación en marcha; pero si es el último paso de la ruta, la orientación final conserva ese pequeño residuo.

13.2 Límites geométricos, visuales y de protocolo

Límite identificado en el código / hardware	Valor o condición actual	Consecuencia operativa y mitigación vigente
Rebase iterativo, sin fondo fijo	Tramos de 200 mm, lateral 400 a 700 mm, seis intentos	Puede ampliar la recta final hasta 2 m. Si no cabe o agota intentos pide AYUDA. El modelo 35×45 y el de fondo 90 cm terminan sin contacto; solo SIMULADO.

Límite identificado en el código / hardware	Valor o condición actual	Consecuencia operativa y mitigación vigente
Objetos del mismo color que el piso y no reconocidos por COCO	Clasificador YCbCr + EfficientDet-Lite0	Si un obstáculo tiene exactamente la misma crominancia que la baldosa y no pertenece a una clase reconocible por la red neuronal, la cámara puede no marcarlo; la defensa es habilitar usar_tof_en_ruta (esquive láser a 400 mm) y el freno local TOF_PARO a 120 mm.
Geometría puntual del láser VL53L0X	Cono frontal de ~25° a la altura del borde delantero	Objetos muy bajos (< 4 cm, por debajo del haz) o muy estrechos desplazados hacia una rueda pueden no reflejar el pulso láser.
Límite de 6 dígitos en el comando RUMBO:<décimas>	Máximo ±99999 décimas (~±10,000° ≈ 27.7 vueltas completas)	En cualquier ruta normal o cuadrado los ángulos se mantienen dentro de ±360°; solo una rotación continua en espiral de más de 27 vueltas sin recalibrar CAL_IMU excedería el formato de 6 caracteres.
Transición a AYUDA **en 1 de 6 corridas físicas B6**	Corrida 5 (J16_B6_r5, paso 4/10)	El vehículo se detuvo de forma segura sin tocar la caja pero solicitó ayuda teniendo espacio físico. Para diagnosticar su causa exacta se requiere un registrador persistente de telemetría por corrida.

Tabla 18. *Resumen estructurado (Límite identificado en el código / hardware · Valor o condición actual).*

13.3 Resumen de elementos NO MEDIDOS y trabajo futuro (PENDIENTE)

Para orientar a futuros equipos o siguientes iteraciones del proyecto, las mejoras pendientes se ordenan por impacto técnico:

1. **Completar la campaña de pruebas físicas en el carro (NO MEDIDO → VALIDADO):**
   - Ejecutar con baterías al 100 % las series físicas de **recta de 150 cm con y sin** RUMBO (B5), **ruta en L** (B7), **cuadrado de 60 cm** (B8), **calibración en piso de** IMU_ESCALA (giro de 360°), **corte inducido de WiFi en marcha** (B9), **freno autónomo aislado por láser** (120 mm) y **obstáculo ancho de** 40×20 cm.
2. **Incorporar odometría de rueda o flujo óptico calibrado (PENDIENTE):**
   - Añadir sensores Hall o codificadores ópticos de ranura en al menos dos ejes de motor TT para medir directamente los milímetros recorridos y independizar la distancia longitudinal del voltaje de las pilas AA.
3. **Reemplazar el puente H doble por cuatro canales independientes (PENDIENTE):**
   - Sustituir el driver de dos canales TC1508A por dos puentes H dobles (4 canales PWM independientes) para aprovechar la cinemática omnidireccional completa de las ruedas *mecanum* (desplazamiento lateral puro sin necesidad de girar 90° para esquivar).
4. **Registrador automático de caja negra por corrida (PENDIENTE):**
   - Guardar en almacenamiento interno del teléfono un archivo .jsonl o .csv por cada ejecución de ruta con todas las tramas SENS, decisiones de fusion.c y estados de navegador.c para diagnosticar con certeza eventos como la parada en AYUDA de la Corrida 5.
5. **Medición eléctrica instrumentada (NO MEDIDO):**
   - Caracterizar con amperímetro el perfil de corriente en arranque (250 ms a PWM 255), avance continuo y bloqueo, así como verificar en banco el circuito externo de GPIO 2.

Dónde está en el código: limites_conocidos.md, estado_20261110.md, reporte_J16.md, navegador.h, config.h.

13.4 Consenso y alcance de la revisión de autonomía

El láser cercano o la coincidencia color/red por zona confirman obstáculo. Con láser libre, una sola fuente no frena. Sin eco se decide por visión; el cierre necesita varios cuadros sin coincidencia y rebase cumplido. No hay fondo fijo de 250 mm: se explora y reintenta. El supuesto sensor-centro de 120 mm y las bandas visuales siguen pendientes de medición.

Banquito 35×45 y arreglo **NO MEDIDOS** hasta repetir primero con caja de 20×20 cm, previa autorización. La app exige Aprender piso, pero no identifica automáticamente todos los

cambios de iluminación. El modelo no cubre todos los objetos ni acredita espacio trasero dinámico. Códigos y presupuestos: `docs/limites_conocidos.md`. No se instaló APK, grabó firmware ni declaró v1.2.

14. Proceso de desarrollo

Este capítulo resume la evolución técnica del proyecto desde el prototipo inicial de control por WiFi hasta la arquitectura distribuida con verificación automatizada y cierre de precisión de la **Versión final 1.1**. Más que enumerar fechas, explica qué problema concreto apareció en cada etapa y por qué se tomaron las decisiones de ingeniería que dieron forma al sistema actual.

14.1 Del control monolítico al desacoplamiento por interfaces en C99

En el punto de partida del proyecto (`original_backup`), un solo archivo de Arduino concentraba la configuración de la red `WiFi`, el servidor web, la recepción de mensajes y la escritura directa en los pines de los motores, mientras que la aplicación Android únicamente mostraba botones básicos. Ese enfoque inicial tenía dos limitaciones severas:

1. **Imposibilidad de probar la lógica sin el carro físico:** cualquier cambio en el manejo de comandos exigía grabar la placa ESP32, encender las baterías y observar manualmente si las ruedas giraban, sin forma de reproducir fallos de temporización o condiciones de frontera.
2. **Vulnerabilidad ante pérdida de conexión o saturación de mensajes:** si el teléfono se desconectaba mientras enviaba una orden de avance, o si los mensajes llegaban desde el hilo asíncrono de red al mismo tiempo que el bucle principal tocaba los pines, el vehículo podía quedar con los motores encendidos.

La primera decisión estructural consistió en extraer toda la lógica del firmware a módulos en estándar `C99` **puro** (`carro.c`) separados del hardware mediante funciones abstractas (`hw_*`), introducir la **cola determinista de 32 ranuras con prioridad absoluta para** `STOP` **y** `EBRAKE`, fijar el *watchdog* de comunicación en `1,000 ms` y crear la suite `pruebas_firmware` junto con el servidor falso de protocolo.

14.2 Creación del núcleo matemático compartido (`nucleo_c`) y puente JNI

Para dotar al vehículo de navegación por rutas sobre una cuadrícula de `20 cm` sin duplicar código entre la computadora de desarrollo y el teléfono Android, se diseñó la biblioteca `nucleo_c` en `C99` sin dependencias externas:

- Se implementaron el planificador geométrico (`trayectoria.c`), el editor de rutas (`ruta_editor.c`), el modelo lineal de conversión distancia-tiempo con compensación de arranque y frenado (`calibracion.c`) y la máquina de estados del navegador (`navegador.c`), enlazándolos con Kotlin mediante `puente.c` (`JNI`).
- Al mantener el núcleo en `C99`, se pudo construir un **simulador de vehículo y entorno (**`simulador_navegador.c`**)** capaz de ejecutar miles de recorridos completos en milisegundos dentro de `make -C nucleo_c test`.

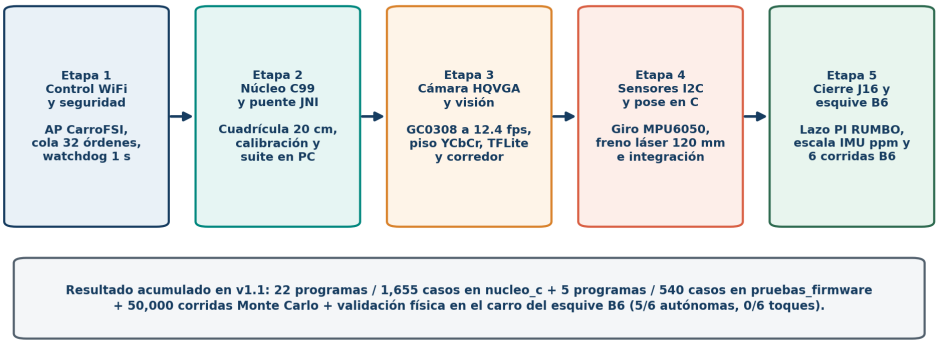


Figura 12. *Evolución técnica del sistema en cinco etapas: desde el control WiFi inicial hasta el cierre de precisión J16 y la validación física del esquive B6.*

14.3 Integración de cámara independiente y percepción multimodal

La incorporación de visión artificial planteó un dilema de rendimiento: el sensor GC0308 de la placa Freenove ESP32-WROVER carece de compresión JPEG por hardware y requiere codificación por software. Un barrido sistemático de resoluciones y frecuencias de reloj demostró que HQVGA **(**240×176 **píxeles) a** 20 MHz reducía la edad media de cuadro a 137 ms (12.4 fps), casi la mitad del rezago de QVGA (252 ms), y el diseño de **buzón de último cuadro** más el modo UDP de celdas (1,244 bytes) eliminaron la acumulación de imágenes viejas en cola.

En el lado de percepción, las pruebas con pisos reales mostraron que ni el color del suelo ni una red neuronal por separado bastaban para todos los objetos. Se construyó entonces la tubería multimodal: clasificador YCbCr de hasta tres clústeres con la regla de tolerancia en filas lejanas (obstaculos.c), máscara de artefactos (mascara_vision.c), red EfficientDet-Lite0 en hilo dedicado con vigencia de 300 ms (ModeloEnHilo.kt), proyección trapezoidal del ancho del carro (corredor.c) y módulo de fusión (fusion.c).

14.4 Sensores embarcados (MPU6050 y VL53L0X) e integración de pose

Las primeras pruebas físicas en pista revelaron dos fenómenos mecánicos propios del chasis de cuatro motores TT:

1. Con ambos lados al máximo (255/255), la asimetría de los motores desviaba el carro 14.5 cm en una recta de 150 cm (corregido inicialmente a 3.0 cm en modo MANUAL ajustando 252/255), y los giros calculados solo por tiempo variaban con la carga de las pilas AA y la textura del piso.
2. En la maniobra de esquive, el navegador proyectaba los avances suponiendo que el carro mantenía siempre el rumbo ideal del tramo (0°, ± 90°), por lo que cualquier error angular durante el rodeo desplazaba el punto de reincorporación.

Para resolver el primer punto se integraron en el bus I2C (GPIO 21/22) el giroscopio MPU6050 (muestreado a 200 Hz con lazo cerrado de giro giro.c a ±2.0°) y el sensor láser VL53L0X (con paro reflejo local a 120 mm en tof.c). Para resolver el segundo punto, se rediseñó el estimador de pose de navegador.c para que integrara cada incremento de avance usando el ángulo real reportado por el giroscopio en SENS y calculara trigonométricamente el vector de regreso hacia la línea original.

14.5 Cierre de precisión J16 y validación física del esquive en el carro

Finalmente, el análisis de precisión y concurrencia condujo al cierre **J16** que culmina esta **Versión final 1.1**:

- Se eliminó la pérdida de precisión por truncamiento integrando la pose en **décimas de grado (**rumbo_x10**)** y corrigiendo override_largo_mm y descartar_dt.
- Se implementó el **controlador** PI **de rumbo en rectas (**rumbo.c**)** con aprendizaje de sesgo entre tramos (aprendido_mil), guiado por punto de mira a 400 mm y supresión de giros detenidos pequeños (≤ 8°).
- Se agregó el **factor de escala del giroscopio en partes por millón (**IMU_ESCALA **en** NVS**)**, la normalización de frenado en giros largos (GIRO_LARGO_POR_GRADO), el candado mutex candado_vision en JNI, el aviso WD:<n> y la vigilancia triple en ModoSemiautonomo.kt.
- Tras verificar el conjunto con **50,000 corridas Monte Carlo** y **2,267 casos de prueba automatizados en PC**, se validó físicamente en el carro el **esquive con reincorporación B6** (6 corridas el 10 de noviembre de 2026: 0/6 toques de caja y 5/6 corridas autónomas completas con error lateral aproximado entre ~0 y ~10 cm).

Dónde está en el código: bitacora_desarrollo.md, decisiones_diseno.md, reporte_J16.md, carro.c, rumbo.c, navegador.c, puente.c.

15. Solución de problemas

Este capítulo proporciona una guía sistemática de diagnóstico y resolución de fallas para los problemas que pueden presentarse durante la preparación, conexión u operación del vehículo **LIDIA**. Sigue el principio de aislar una sola variable a la vez, comenzando siempre por la alimentación eléctrica y las conexiones físicas con los motores apagados.

15.1 Método general de aislamiento de fallas

Cuando ocurre un comportamiento inesperado en el sistema, modificar simultáneamente cables, baterías y ajustes de la aplicación impide identificar la causa raíz. El procedimiento recomendado consta de cuatro pasos:

1. **Inmovilizar el vehículo:** pulsar `PARO` en la aplicación o pasar el interruptor del portapilas `4×AA` a `OFF` y elevar el chasis antes de revisar el cableado.
2. **Separar los tres enlaces:** comprobar primero si existe la red `CarroFSI` (`192.168.4.1`), después si responde el control por `WebSocket` (`ESTADO` / `SENS?`) y por separado si llega video desde `192.168.4.2`.
3. **Consultar la telemetría interna:** abrir `Más → Sensores` (o el monitor serie a `115,200` baudios) para leer las banderas de `SENS:<ang_x10>,<mm>,<flags>` y `RUMBO?`.
4. **Cambiar una variable por vez:** tras ajustar un cable, recargar una batería o recalibrar el piso, repetir una prueba corta y registrar el resultado.

15.2 Tabla maestra de diagnóstico por síntoma

Síntoma observado	Causa probable	Verificación inmediata	Solución paso a paso
1. No aparece la red WiFi `CarroFSI` en el teléfono	Powerbank #1 descargada, cable micro-USB de solo carga flojo o ESP32 `DevKit` sin arrancar.	Revisar LED de encendido en la `ESP32 DevKit` y carga de la Powerbank #1.	Conectar la Powerbank #1 con un cable firme, presionar el botón `EN`/`RST` de la `ESP32 DevKit` y esperar `3 s`.
2. Conectado a `CarroFSI`, pero la app dice `Reconectando` o sin control	El teléfono redirige el tráfico por datos móviles al ver que `CarroFSI` no tiene internet, o hay otro cliente conectado.	Abrir en el navegador `http://192.168.4.1/` o revisar si está activo el cambio inteligente de red en Android.	Desactivar datos móviles o aceptar `"Mantener conexión WiFi sin internet"` en Android; verificar que la IP en `Ajustes` sea `192.168.4.1`.
3. Hay control del carro, pero no se ve video de la cámara	Powerbank #2 (propia de la cámara) apagada, cámara aún no asociada a `CarroFSI` o segundo cliente bloqueando `/stream`.	Verificar LED en la placa `Freenove` y abrir `http://192.168.4.2/stats` desde el teléfono.	Encender la Powerbank #2 **después** de que la `ESP32 DevKit` ya creó `CarroFSI`; cerrar cualquier pestaña de navegador que tenga abierto `:81/stream`.
4. La app envía órdenes (`OK`), pero ninguna rueda gira	Interruptor del portapilas `4×AA` en `OFF`, pilas agotadas o falta el cable de tierra común (`GND`).	Medir o revisar el portapilas `4×AA` y la unión `GND` entre `DevKit`, `TC1508A` y pilas.	Pasar el interruptor a `ON`, comprobar que el cable `GND` común esté firme en la protoboard y usar 4 pilas `AA` con carga completa.
5. En modo `AUTO`, el botón `Adelante` no avanza y muestra `TOF_PARO`	El sensor láser frontal `VL53L0X` detecta un objeto o la mano del operador a `< 120 mm` (bandera `flags & 16`).	Abrir `Más → Sensores` y leer la distancia en milímetros (`< 120 mm`).	Retirar el obstáculo frontal hasta que la lectura supere `150 mm` (umbral de histéresis `TOF_LIBRE_MM`) o usar `Atrás`/`Izquierda`/`Derecha`.
6. El ángulo en `Sensores` deriva rápido con el carro quieto o devuelve `CAL_ERR_MOV`	El vehículo fue movido durante el primer segundo tras encenderse o la mesa/piso vibra al pulsar `Calibrar IMU`.	Observar si la bandera `imu_calibrada` (`bit 1`, valor 2) está en `0` dentro de `flags`.	Apoyar el carro en el piso firme sin tocarlo, abrir `Más → Sensores`, pulsar `Calibrar IMU` (`CAL_IMU`) y esperar `1 s` hasta ver `CAL_OK`.

Síntoma observado	Causa probable	Verificación inmediata	Solución paso a paso
7. El carro se curva sistemáticamente hacia un lado en rectas de ruta	Control RUMBO desactivado (OFF), pilas AA descargadas o rueda con roce mecánico.	En Más → Sensores, pulsar RUMBO? y comprobar que devuelva RUMBO:1,... (on=1).	Activar el interruptor Control de rumbo en rectas (RUMBO:ON), verificar que ninguna llanta roce con cables y recalibrar CAL_IMU antes de salir.
8. En giros acumulados (90° o 360°) el carro gira sistemáticamente de más o de menos	Tolerancia de sensibilidad del giroscopio MEMS (±1 % a ±3 %) con escala_ppm = 0.	Consultar IMU_ESCALA? en Sensores y comparar un giro físico de 360° contra el ángulo reportado.	Ejecutar en Más → Sensores el asistente de 3 pasos (1. Poner a 0° → girar 360° físicos → 3. Calcular y guardar) para persistir IMU_ESCALA en NVS.
9. La cámara frena en falso en piso despejado o ignora la caja en ruta	El modelo de color de suelo corresponde a otra habitación o cambió la iluminación ambiental.	Abrir Más → Visión y observar si el trapecio del corredor marca celdas rojas sobre el piso limpio.	Apuntar el carro hacia un tramo despejado del piso actual con la iluminación de trabajo y pulsar Aprender piso.
10. La ruta se detiene sola a los 1.0 o 1.5 s con aviso Watchdog o Sin datos de sensores	El teléfono entró en suspensión de pantalla, se alejó fuera de cobertura WiFi o sufrió interferencia severa.	Revisar el mensaje en pantalla (WD:<n> o silencio de SENS > 1.5 s).	Mantener la pantalla del teléfono encendida en primer plano y caminar a menos de 2–3 m del vehículo durante toda la ruta.

Tabla 19. *Resumen estructurado (Síntoma observado · Causa probable).*

15.3 Verificación rápida por puerto serie (cuando no hay WiFi disponible)

Si se desea diagnosticar la placa ESP32 DevKit directamente desde una computadora por cable USB sin usar el teléfono, basta abrir cualquier terminal serie a 115,200 **baudios** (terminador de línea \n) y enviar los cuatro comandos de inspección:

1. VERSION? → debe responder FW:6d1b213-20261110.
2. ESTADO → debe responder AUTO (o MANUAL) seguido de CAPS:IMU=1,CAL=1,TOF=1,GIRO=1,RUMBO=1,IMUS=...,FW:6d1b213-20261110.
3. SENS? → debe responder SENS:<ang_x10>,<mm>,7 (donde flags = 7 confirma que tanto el MPU6050 en 0x68 como el VL53L0X en 0x29 están respondiendo en el bus I2C).
4. RUMBO? → debe responder RUMBO:1,0,0,0,0,0 (confirmando que el controlador PI de rumbo está habilitado y listo).

Dónde está en el código: carro.c, sensores.c, recuperacion.c, Enlace.kt, ModoSemiautonomo.kt, SensoresActivity.kt.

16. Anexos

Este capítulo reúne las referencias de consulta rápida del proyecto: (16.1) el glosario de términos técnicos, (16.2) el resumen completo de comandos del protocolo y de pines de hardware, (16.3) la tabla de correspondencia entre los capítulos de este manual y las rutas exactas de archivos en el repositorio, y (16.4) la lista consolidada de mediciones físicas y tareas pendientes.

16.1 Glosario técnico

- **Anti-*windup* (saturación de integrador):** técnica de control que impide que el término integral de un controlador PI siga acumulando error cuando el actuador ya alcanzó su límite físico (±150 milésimas en `rumbo.c`).
- **Buzón de último cuadro (*latest-frame mailbox*):** estructura de memoria compartida en la cámara Freenove que conserva únicamente el cuadro más reciente capturado por el sensor, descartando cuadros anteriores no consumidos para evitar latencia acumulada.
- **Corredor de perspectiva:** proyección trapezoidal sobre la imagen (`corredor.c`) que delimita la franja exacta del piso por la que pasarán las ruedas del carro (`150 mm` de ancho físico más `50 mm` de margen por lado).
- **Cuerda de pivote (`2Rsin(Δθ/2)`):** desplazamiento lineal que experimenta un punto del chasis situado a una distancia `R` del centro instantáneo de rotación cuando el vehículo gira un ángulo `Δθ`.
- **Décimas de grado (`rumbo_x10` / `ang_x10`):** unidad angular entera empleada en la telemetría `SENS`, en el controlador `rumbo.c` y en el estimador de pose de `navegador.c` (`10` unidades equivalen a `1.0°`), que evita el uso de punto flotante en el microcontrolador sin perder resolución por truncamiento.
- **Escala del giroscopio en ppm (`escala_ppm` / `IMU_ESCALA`):** corrección multiplicativa en partes por millón (`10,000 ppm = 1.0 %`) almacenada en la memoria NVS de la ESP32 para compensar el error de sensibilidad de fábrica del `MPU6050`.
- **Histéresis de desbloqueo (`TOF_LIBRE_MM = 150 mm`):** diferencia deliberada entre el umbral que activa el freno láser (`120 mm`) y el umbral que permite volver a avanzar (`150 mm`), necesaria para evitar oscilaciones de encendido/apagado frente a un obstáculo cercano.
- **JNI (*Java Native Interface*):** interfaz binaria (`puente.c`) que permite a la aplicación Android en Kotlin invocar directamente las funciones compiladas en C99 de `nucleo_c` protegidas por `candado_vision`.
- **NVS (*Non-Volatile Storage*):** partición de memoria flash interna de la ESP32 donde el firmware conserva parámetros entre apagados, como `escala_ppm` del giroscopio.
- **Punto de mira (*lookahead* de `400 mm`):** punto objetivo situado `40 cm` por delante sobre el segmento ideal de la ruta hacia el cual `navegador.c` orienta continuamente la referencia `RUMBO:<décimas>`.
- **Sesgo aprendido (`aprendido_mil`):** valor de compensación diferencial de PWM (acotado a ±120 milésimas) que el controlador PI de `rumbo.c` transfiere al finalizar un tramo recto para arrancar el siguiente tramo ya compensado.
- **ToF (*Time of Flight*, tiempo de vuelo):** principio físico del sensor láser `VL53L0X`, que determina la distancia en milímetros midiendo el tiempo que tarda un pulso de luz infrarroja (`940 nm`) en ir hasta el objeto y regresar al fotodiodo.
- **Watchdog de comunicación (`1,000 ms`):** temporizador de seguridad en `carro.c` que apaga los motores y emite `WD:<n>` si transcurre un segundo sin recibir órdenes de movimiento válidas.

16.2 Referencia rápida de comandos y pines

Resumen compacto de comandos del protocolo (`ws://192.168.4.1/ws` y serie `115200`)

Categoría	Comando	Respuesta normal	Descripción rápida
Tracción	FORWARD, BACKWARD, LEFT, RIGHT	OK *(o* BLOQUEADO_TOF*)*	Mueve el carro y renueva el *watchdog* de 1000 ms (latido cada 250 ms).
Parada	STOP / EBRAKE	OK / OK_EBRAKE	Paro libre (LOW/LOW) o freno activo (HIGH/HIGH) con prioridad en cola.
Perfil	MODO_AUTO / MODO_MANUAL / ESTADO	ACK_AUTO / ACK_MANUAL / AUTO+CAPS	Selecciona PWM 207/210 (AUTO) o 252/255 (MANUAL) y reporta capacidades.
Enlace y versión	PING / VERSION? / LOOP?	PONG / FW:6d1b213-20261110 / LOOP:...	Sonda de red (800 ms), revisión compilada y tiempos de bucle en microsegundos.
Sensores e IMU	SENS? / CAL_IMU	SENS:<ang_x10>,<mm>,<flags> / CAL_OK	Telemetría en décimas de grado y milímetros (10 Hz en AUTO) y calibración en reposo.
Giro y escala	GIRO:<g> / IMU_ESCALA:<ppm> / IMU_ESCALA?	GIRO_FIN:<a> / IMU_ESCALA_OK:<ppm>	Giro cerrado en [-360,360] (±2.0°) y factor de escala en [-50000,50000] ppm en NVS.
Rumbo en rectas	RUMBO:ON / RUMBO:OFF / RUMBO:<d> / RUMBO?	RUMBO_OK:... / RUMBO_REF:<d> / RUMBO:...	Activa/desactiva el lazo PI en rectas, fija referencia en décimas de grado o consulta estado.

Tabla 20. *Resumen estructurado (Categoría · Comando).*

Resumen de pines en la placa ESP32 DevKit y direcciones de red

Recurso	Pin / Dirección	Función asignada	Parámetros clave
Motor izquierdo (IN1, IN2)	GPIO 27, GPIO 26	Canal 1 del puente H TC1508A	PWM 1000 Hz, 8 bits (MANUAL=252, AUTO=207).
Motor derecho (IN3, IN4)	GPIO 25, GPIO 33	Canal 2 del puente H TC1508A	PWM 1000 Hz, 8 bits (MANUAL=255, AUTO=210).
Bus I2C (SDA, SCL)	GPIO 21, GPIO 22	MPU6050 (0x68) y VL53L0X (0x29)	Reloj 400 kHz, alimentación 3.3 V.
Carga de *powerbank*	GPIO 2	Pulso de 200 ms cada 8,000 ms	Declarado en config.h; efecto físico NO MEDIDO.
Red CarroFSI (carro / cámara)	192.168.4.1 / 192.168.4.2	Control :80/ws; video :81/stream, :80/raw, UDP :4210	Un cliente de video simultáneo; paquetes CEL1 de 1,244 B.

Tabla 21. *Resumen estructurado (Recurso · Pin / Dirección).*

16.3 Correspondencia entre capítulos y archivos del repositorio

Esta tabla concentra todas las rutas de archivos del repositorio correspondientes a cada capítulo del manual:

Capítulos	Subsistema	Rutas exactas de archivos en el repositorio
Cap. 1 y 2	Documentación general, arquitectura y seguridad	README.md, LICENSE, docs/README.md, docs/entrega.md, docs/arquitectura.md, docs/seguridad_fisica.md, docs/estado_20261110.md, docs/limites_conocidos.md, docs/reporte_J16.md.
Cap. 3 y 4	Hardware y firmware del carro (ESP32 DevKit)	firmware_carro/firmware_carro.ino, firmware_carro/config.h, firmware_carro/secretos.ejemplo.h, firmware_carro/pagina.h, firmware_carro/carro.h, firmware_carro/carro.c, firmware_carro/sensores.h, firmware_carro/sensores.c, firmware_carro/rumbo.h, firmware_carro/rumbo.c, firmware_carro/imu.h, firmware_carro/imu.c, firmware_carro/giro.h, firmware_carro/giro.c, firmware_carro/tof.h, firmware_carro/tof.c.

Capítulos	Subsistema	Rutas exactas de archivos en el repositorio
Cap. 5	Firmware de cámara (Freenove GC0308)	camara_prueba/camara_prueba.ino, camara_prueba/recuperacion.h, camara_prueba/recuperacion.c, camara_prueba/secretos.ejemplo.h, camara_prueba/src/JPEGENC/, nucleo_c/paquete_celdas.h, nucleo_c/paquete_celdas.c.
Cap. 6	Protocolo y pruebas de enlace	docs/protocolo.md, pruebas_firmware/Makefile, pruebas_firmware/pruebas.c, pruebas_firmware/pruebas_sensores.c, pruebas_firmware/pruebas_rumbo.c, pruebas_firmware/pruebas_recuperacion.c, pruebas_firmware/pruebas_integracion.c, pruebas_firmware/probar_servidor_falso.py.
Cap. 7	Núcleo de navegación, calibración y simulación en C99	nucleo_c/Makefile, nucleo_c/navegador.h, nucleo_c/navegador.c, nucleo_c/trayectoria.h, nucleo_c/trayectoria.c, nucleo_c/calibracion.h, nucleo_c/calibracion.c, nucleo_c/calgiro.h, nucleo_c/calgiro.c, nucleo_c/ruta_editor.h, nucleo_c/ruta_editor.c, nucleo_c/simulador_navegador.h, nucleo_c/simulador_navegador.c, nucleo_c/sim_ruta.h, nucleo_c/sim_ruta.c, nucleo_c/pruebas_precision_j16.c, nucleo_c/pruebas_reincorporacion_j15.c.
Cap. 8	Visión, corredor y puente JNI	nucleo_c/obstaculos.h, nucleo_c/obstaculos.c, nucleo_c/corredor.h, nucleo_c/corredor.c, nucleo_c/fusion.h, nucleo_c/fusion.c, nucleo_c/cualquiera.h, nucleo_c/cualquiera.c, nucleo_c/mascara_vision.h, nucleo_c/mascara_vision.c, nucleo_c/movimiento_piso.c, nucleo_c/movimiento_visual.c, app_android/app/src/main/cpp/puente.c, app_android/app/src/main/cpp/CMakeLists.txt, app_android/app/src/main/assets/efficientdet_lite0.tflite.
Cap. 9	Aplicación Android (Kotlin y vistas)	app_android/app/build.gradle.kts, app_android/app/src/main/AndroidManifest.xml, app_android/app/src/main/java/com/example/carro_ilib/MainActivity.kt, InicioFragment.kt, ManualFragment.kt, RutasFragment.kt, EjecutarRutaFragment.kt, MasFragment.kt, AjustesFragment.kt, VisionActivity.kt, CalibracionActivity.kt, CalibrarGiroActivity.kt, SensoresActivity.kt, BenchmarkActivity.kt, ControlLogica.kt, ModoSemiautonomo.kt, CalibracionGuardada.kt, SesionCarro.kt, SesionVideo.kt, Enlace.kt, DetectorTflite.kt, ModeloEnHilo.kt, *Nativa.kt.
Cap. 10 a 15	Scripts, artefactos dist/, pruebas físicas y generador del manual	scripts/cargar.ps1, scripts/compilar_app.ps1, scripts/verificar_version.py, scripts/probar_version.py, scripts/verificar_pagina.py, dist/carro_fsi_0.4-6d1b213.apk, dist/SHA256SUMS.txt, dist/FIRMWARE_LOCAL.txt, dist/LEEME.txt, docs/pruebas_fisicas_20261110.csv, docs/precision/monte_carlo_1000.txt, docs/precision/calibracion_j14.txt, docs/figuras/generar_figuras.py, docs/manual/generar_manual.py, docs/manual/Manual_Proyecto_Carro_FSI.pdf.

Tabla 22. *Resumen estructurado (Capítulos · Subsistema).*

16.4 Lista consolidada de pruebas y mediciones pendientes

Identificador	Descripción de la medición o mejora pendiente	Estado actual	Prioridad sugerida
P-01 (B5)	Medir con cinta métrica en el carro físico la recta de `150 cm` con `RUMBO:OFF` vs `RUMBO:ON` (5 corridas por modo con pilas al 100 %).	`SIMULADO` / `NO MEDIDO` en carro	Alta
P-02 (B7 / B8)	Ejecutar en el carro físico la ruta en L (`60×60 cm`) y el cuadrado de `60 cm` con el cierre J16 completo.	`SIMULADO` / `NO MEDIDO` en carro	Alta
P-03 (`IMU_ESCALA`)	Realizar la calibración física de `360°` en piso desde `SensoresActivity` y medir la mejora de cierre angular en el cuadrado.	`VERIFICADO` en código / `NO MEDIDO` en piso	Media
P-04 (B9 / `TOF`)	Medir físicamente la distancia de paro ante corte inducido de WiFi en marcha y en serie aislada de freno láser a `120 mm`.	`VERIFICADO` en software / `NO MEDIDO` aislado	Media
P-05 (Telemetría)	Añadir exportación automática de bitácora `.csv`/`.jsonl` por corrida en Android para diagnosticar transiciones a `AYUDA` como la Corrida 5 de B6.	`PENDIENTE`	Media
P-06 (Hardware)	Medir con amperímetro el consumo de arranque y régimen de los 4 motores `TT` y verificar en banco el circuito externo de `GPIO 2`.	`NO MEDIDO`	Baja

Tabla 23. *Resumen estructurado (Identificador · Descripción de la medición o mejora pendiente).*